STM32最小核心板F103串口通信

一、串口协议、RS-232 标准及 CH340 USB/TTL 转 232 模块原理

1.串口通讯协议

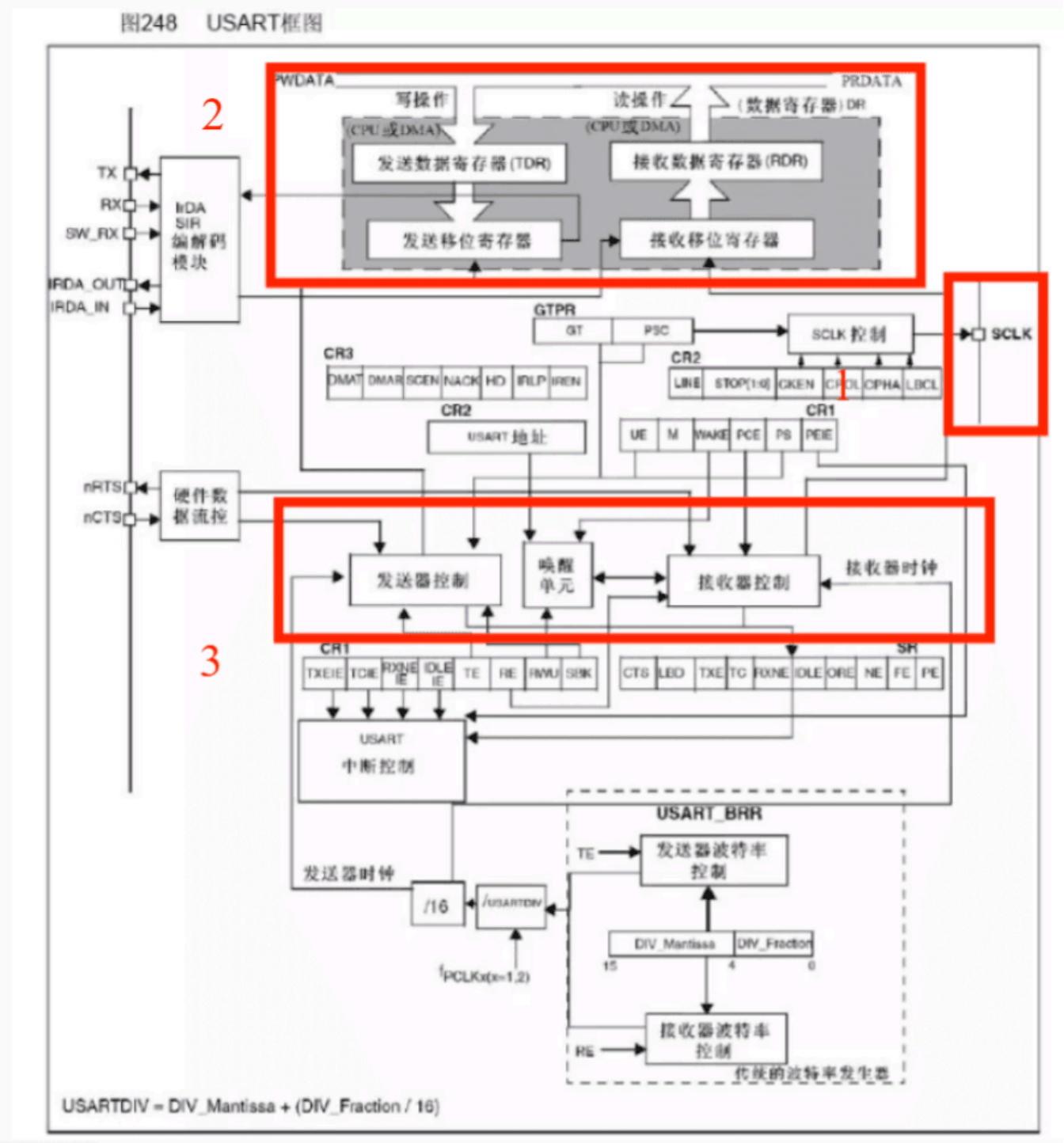
（1）串口协议（UART）基础

ART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）是一种**双向、串行、异步**的通信总线，仅用一根数据接收线和一根数据发送线就能实现全双工通信。

典型的串口通信使用 **3 根线**：发送线（TX）、接收线（RX）和地线（GND），通信时必须将双方的 **TX 和 RX 交叉连接**，并且 **GND 相连** 才能正常通信。

（2）核心特性：

- **异步传输**：不使用时钟信号同步收发设备，而是以约定的**波特率**（如 9600、115200 等）来管理时序。收发双方的波特率差异允许最大值为 **10%**，超过此值位的时序就会脱节。
- **数据包结构**：每个数据包由**起始位**（高电平拉到低电平，保持 1 个时钟周期）、**数据帧**（5～8 位实际数据，最低有效位优先）、**奇偶校验位**（可选）和**停止位**（低电平驱动回高电平，保持 1～2 位时间）组成。



2.RS-232 标准

（1）**RS-232-C** 是美国电子工业协会于 1970 年制定的串行通信接口标准，全名为“数据终端设备（DTE）和数据通讯设备（DCE）之间串行二进制数据交换接口技术标准”。

（2）**关键电气特性：**

逻辑状态	电压范围（发送端）	电压范围（接收端识别）
逻辑“1”（Mark）	-5V ~ -15V	≤ -3V
逻辑“0”（Space）	+5V ~ +15V	≥ +3V

RS-232 采用**负逻辑**关系，噪声容限为 **2V**，即接收器能识别低至 **+3V** 的信号作为逻辑“0”，高到 **-3V** 的信号作为逻辑“1”。最大开路电压为 25V。

RS-232 接口通常使用 **DB-25** 或 **DB-9** 连接器。在实际计算机与终端通信中，一般只使用 **3 条线**：发送数据、接收数据和信号地。标准规定在码元畸变小于 4% 时传输电缆长度应为 **50 英尺**，但实际应用中通常会超过此距离。

3.RS-485 标准

RS-485（TIA/EIA-485A）是一种**差分信令标准**，定义了驱动器和接收器的电气特性，用于实现平衡的多点传输线。

（1）**关键电气特性：**

逻辑状态	AB 两线间电压差
逻辑“1”	**+（2～6）V**
逻辑“0”	**-（2～6）V**

RS-485 采用**差分传输方式**，通过 A、B 两根线的电压差作为电平信号。差分信号能有效抵御外界干扰，因为干扰对两根线的影响是一样的，两根线的电压差不变，信号传递就不会受干扰。

（2）**主要优势：**

- **抗共模干扰能力强**：外部噪声作为共模噪声被差分接收器拒绝，两条信号线的反向开关产生的电磁场相互抵消。
- **传输距离远**：可达 **1200 米**，数据速率最高可达 **50 Mbps。**
- **多点连接**：总线上允许连接多达 **128 个收发器**，而 TTL 和 RS-232 只能一对一连接。
- **共模范围**：驱动器和接收器需在 **-7V 到 +12V** 的共模范围内工作。

RS-485 支持**半双工**（共用一对双绞线）和**全双工**（两对双绞线）两种拓扑，推荐采用**菊花链结构**组网。

4.RS-232、RS-485 与 TTL 电平的区别

特性	TTL	RS-232	RS-485
逻辑“1”	2.4V～5V（5V 系统）	-5V ～ -15V	AB 线差 +2～+6V
逻辑“0”	0V～0.5V	+5V ～ +15V	AB 线差 -2～-6V
传输方式	单端	单端	差分
通信方式	全双工	全双工	半双工 / 全双工
传输距离	< 0.5 米	约 15 米（标准）	可达 1200 米
抗干扰能力	弱	中等	强
连接设备数	1 对 1	1 对 1	最多 128 个

TTL 指双极型三极管逻辑电路，0 对应 0V，1 对应 3.3V 或 5V，与单片机、SOC 的 IO 电平兼容。市面上很多“USB 转 TTL”模块，实际上是“USB 转 TTL 电平的串口”模块。

RS-232 的逻辑电平和 TTL 不一样，但**协议是一样的**——RS-232 用更高的电压和负逻辑来增强抗干扰能力和传输距离。

RS-485 为了长距离传输采用差分方式，传输的是差分信号，即通过 AB 两根线的电压差作为电平信号。从单片机软件编程角度来说，RS-232、RS-485 最终结果都是转换为 TTL 电平方式与单片机通信，目的都是提高通信质量

5.USB/TTL转232"模块的工作原理

这里以CH340芯片模块为例

（1）CH340芯片

CH340 是一个 **USB 总线的转接芯片**，实现 USB 转串口或 USB 转打印口。在串口方式下，CH340 提供常用的 MODEM 联络信号，用于为计算机扩展异步串口，或者将普通的串口设备直接升级到 USB 总线。

主要技术特性：

- 全速 USB 设备接口，兼容 **USB V2.0**
- 硬件**全双工串口**，内置收发缓冲区，波特率范围 **50bps ~ 2Mbps**
- 支持 RTS、DTR、DCD、RI、DSR、CTS 等 MODEM 联络信号
- 支持 **5V 和 3.3V** 电源电压
- 通过**外加电平转换器件**，可提供 RS-232、RS-485、RS-422 等接口
- 内置固件，软件兼容 CH341，可直接使用其 VCP 驱动程序

二、准备工作

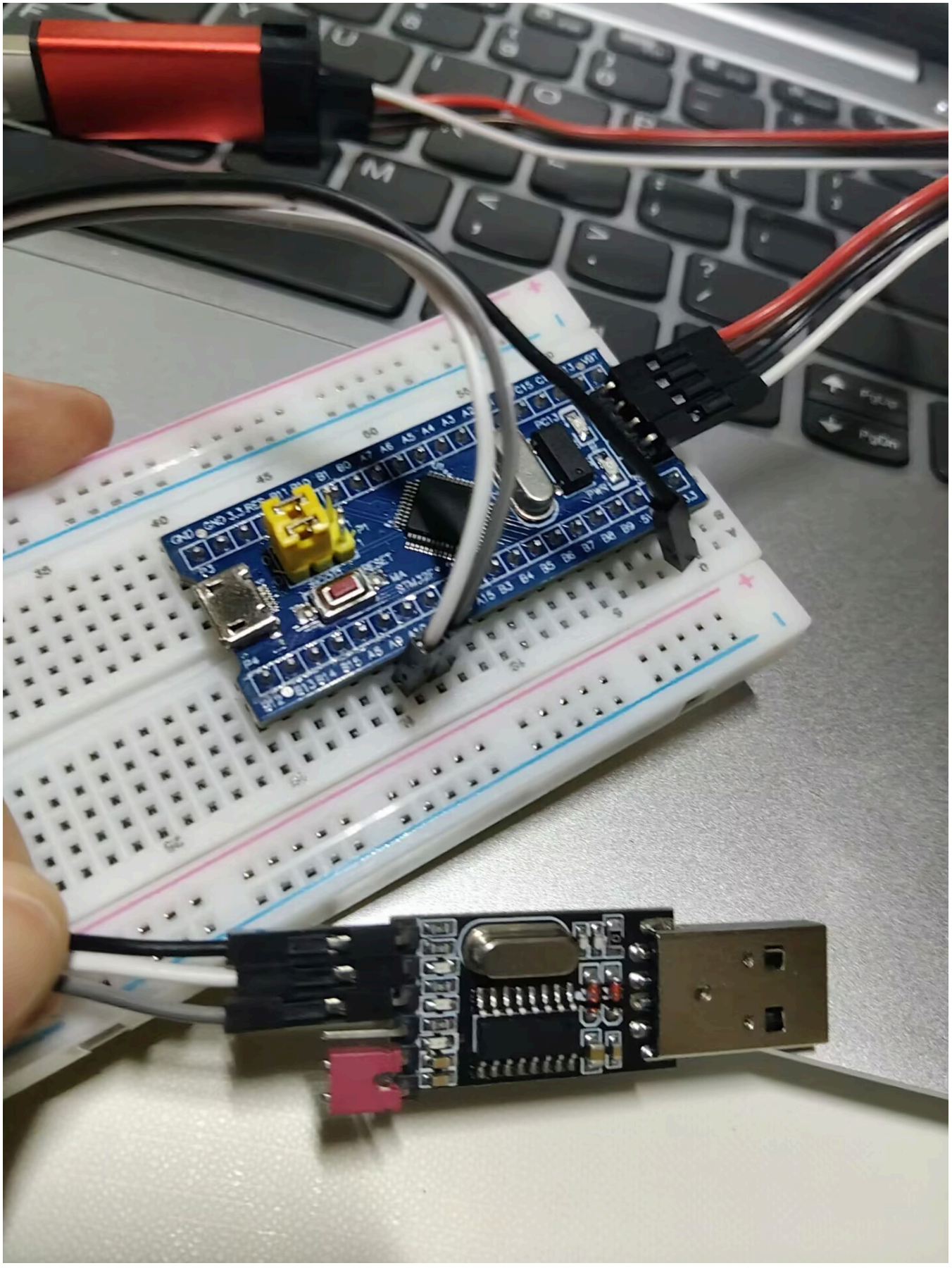
1.sscom串口助手的安装

运行安装



2.硬件连接

USB转串口对应接口：RXD-A9；TXD-A10；GND-GND



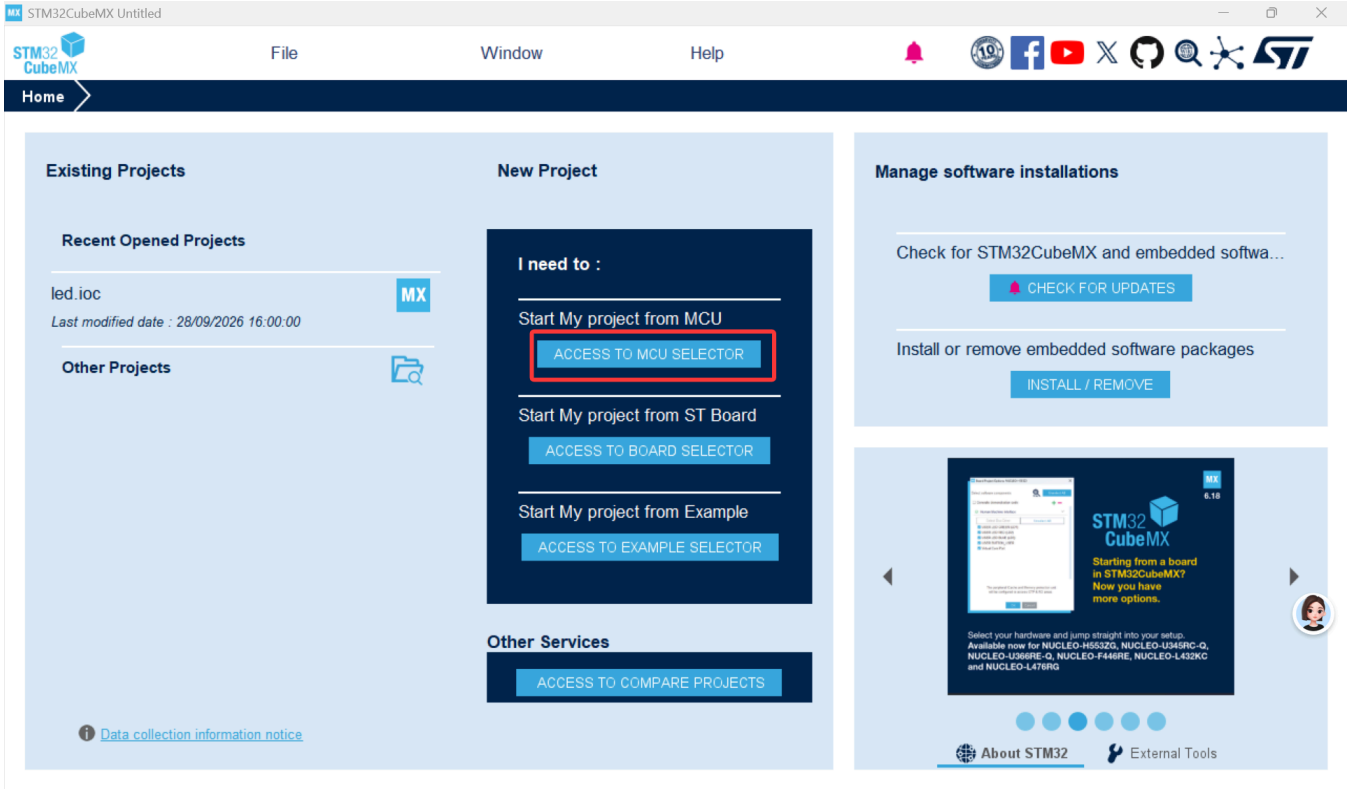
三、任务一

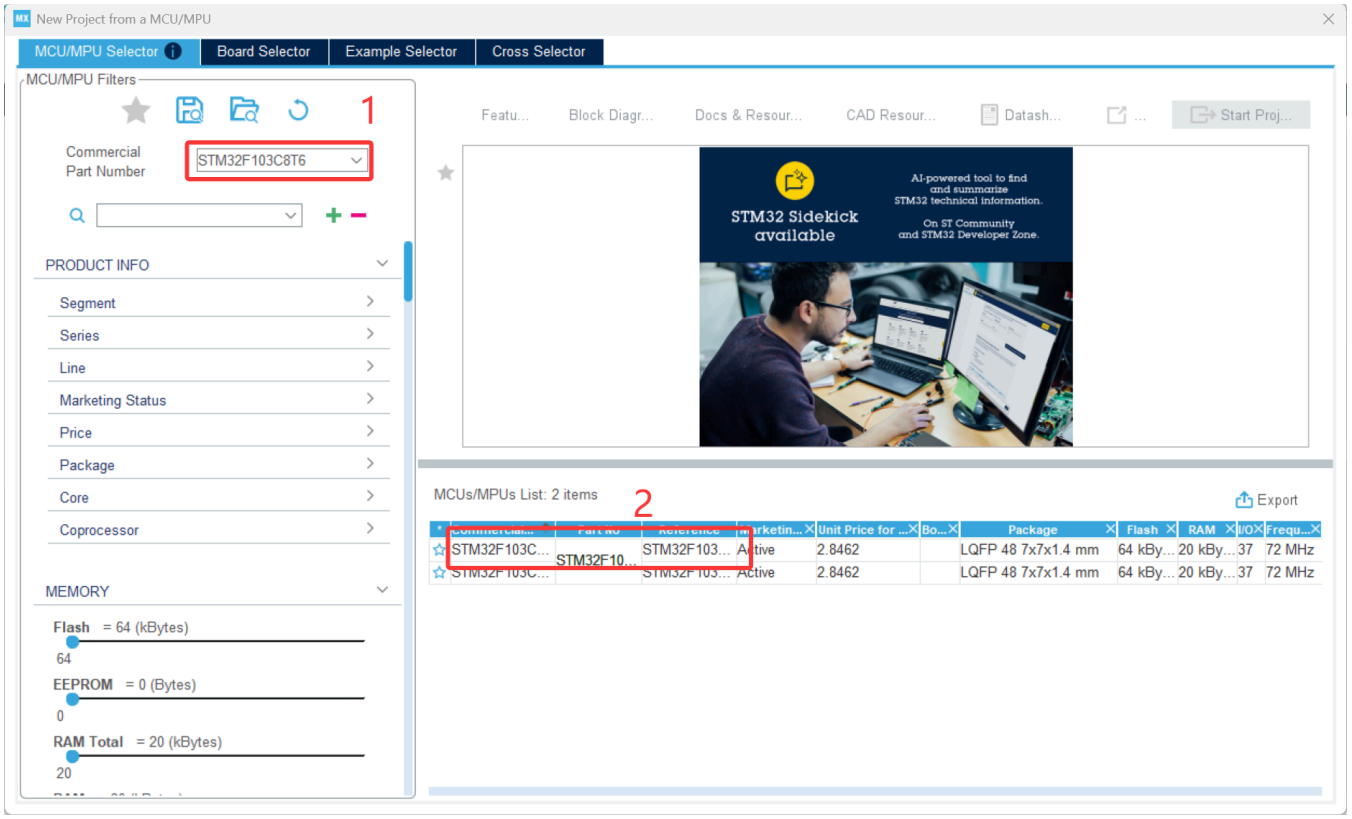
1.实验目的

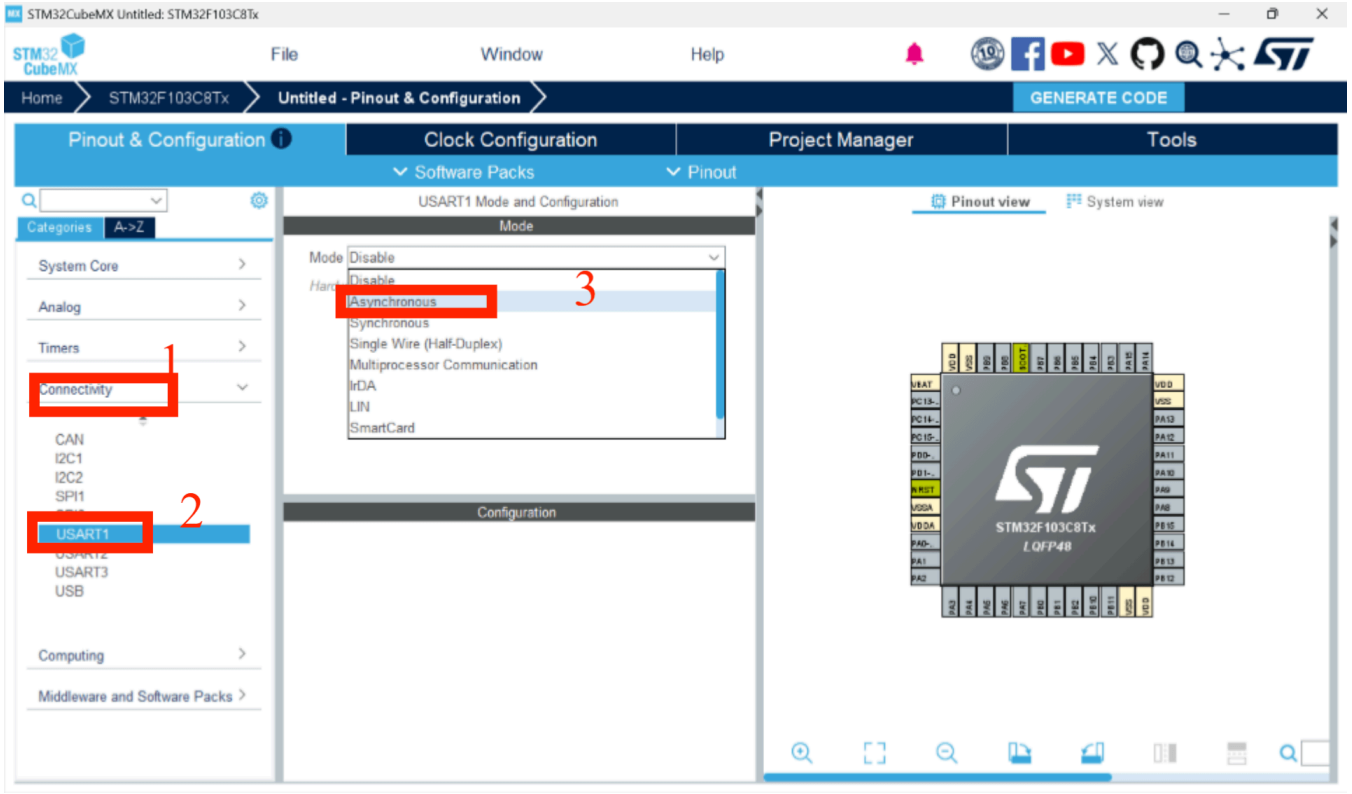
stm32CubeMX，配合Keil，使用HAL库（或标准库）方式，设置USART1 波特率为115200，1位停止位，无校验位，完成下列任务：（1）STM32系统给上位机（win10）连续发送“hello windows！”，win10采用“串口助手”工具接收。如果STM32的连续发送之间不加延时语句，观察win10端是否出现接收数据丢失的现象。（2）在完成以上任务基础，继续扩展功能：当上位机给stm32发送一个字符“#”后，stm32暂停发送“hello windows！”；发送一个字符“*”后，stm32继续发送；

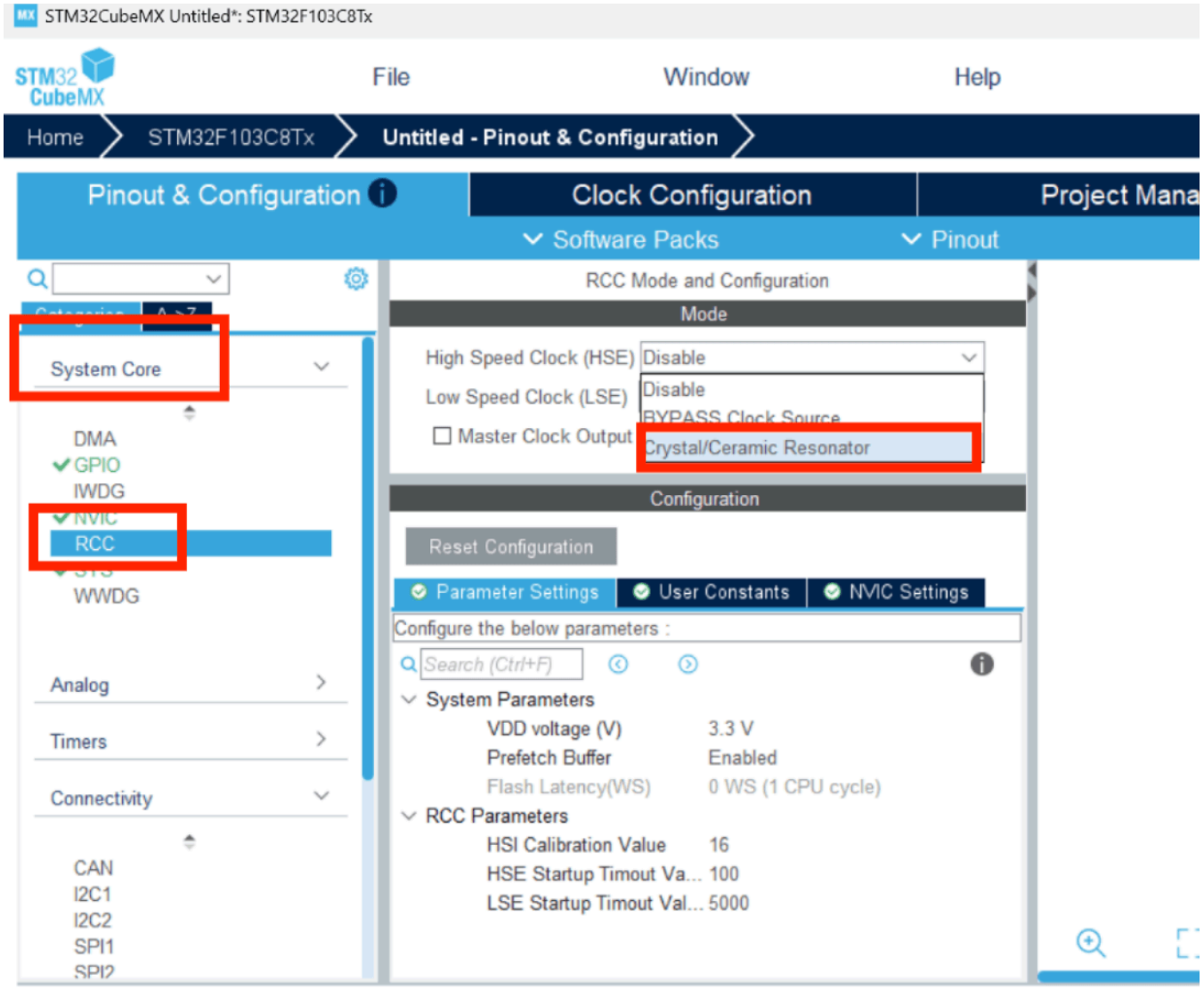
2.工程实现

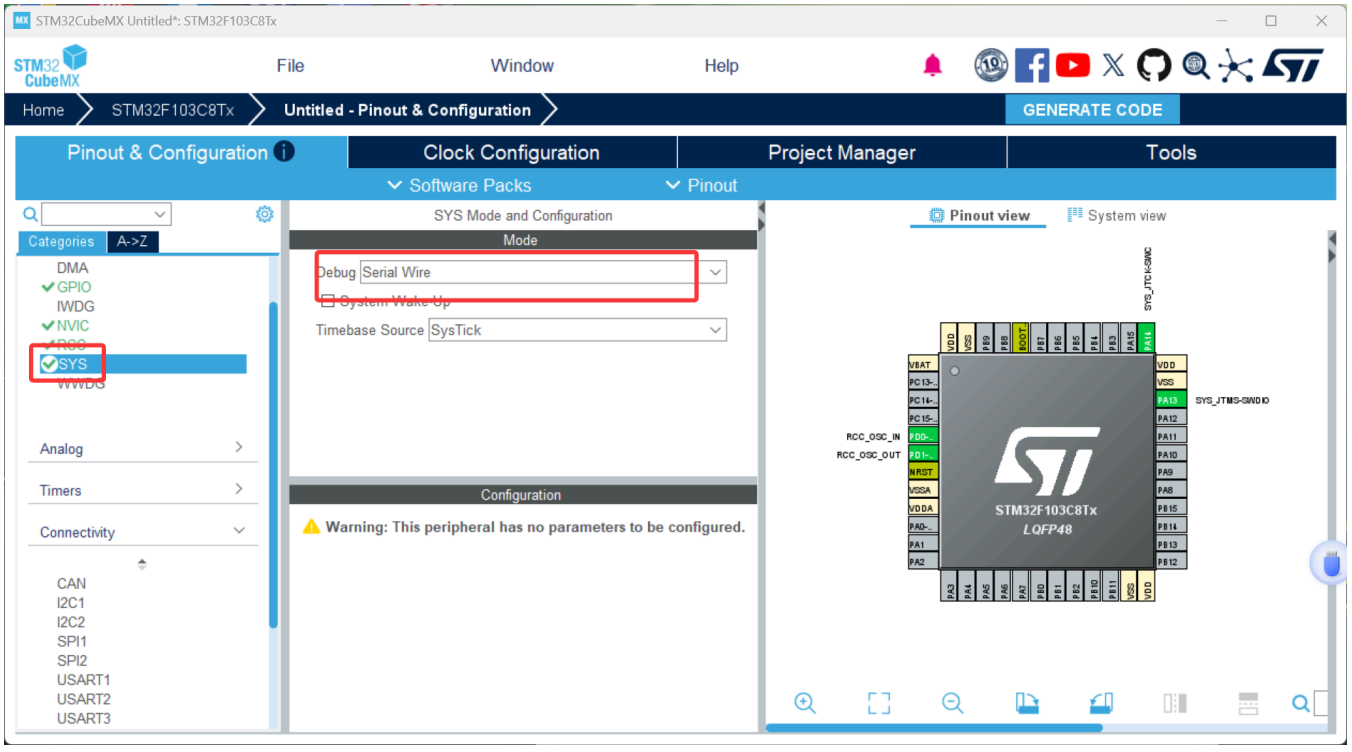
（1）创建项目

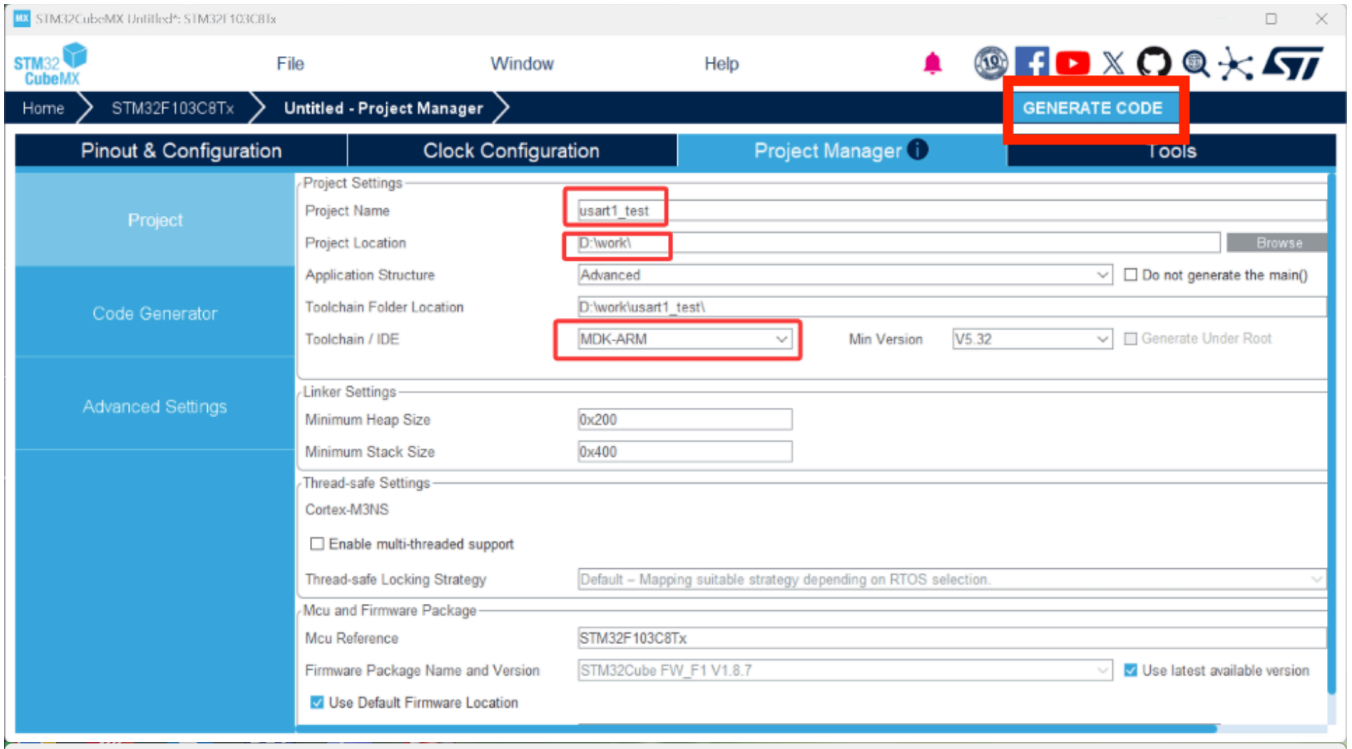


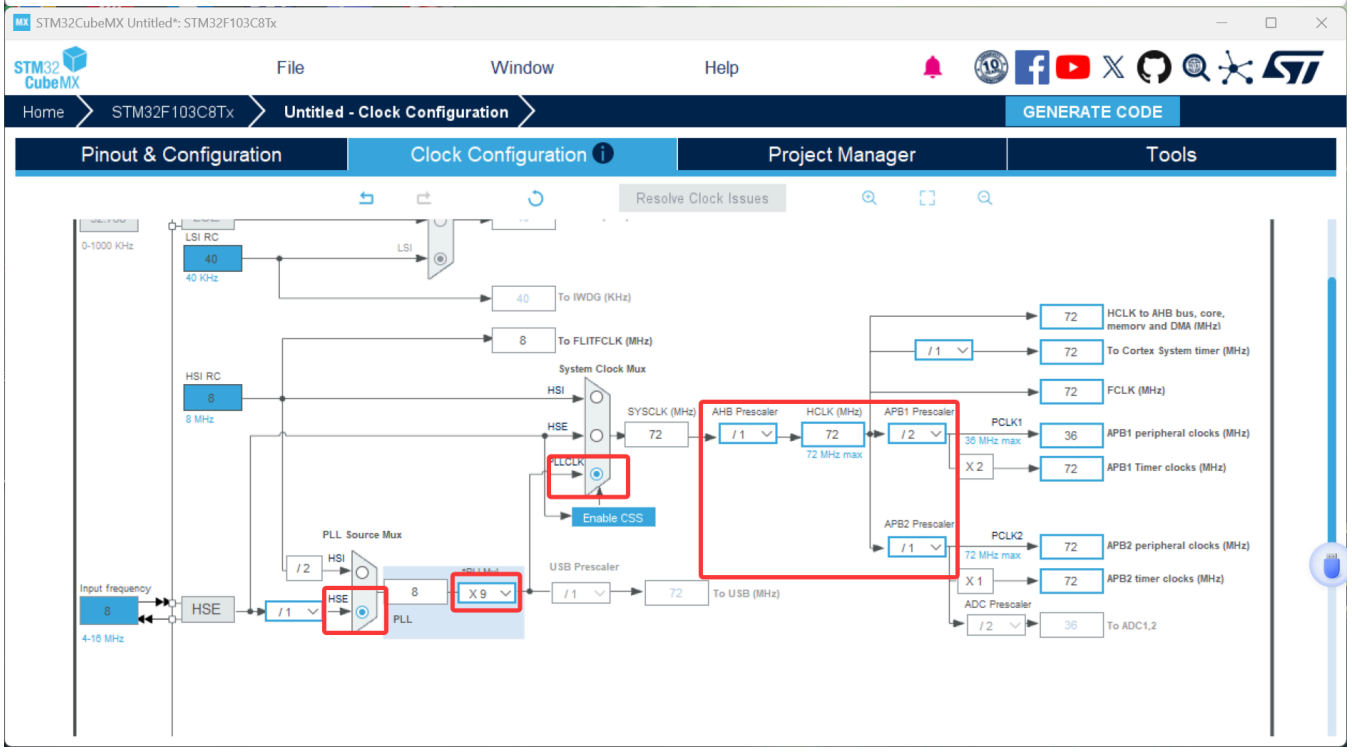

New Project from a MCU/MPU
MCU/MPU Selector
Board Selector
Example Selector
Cross Selector
MCU/MPU Filters
1
Commercial Part Number
STM32F103C8T6
PRODUCT INFO
Segment
Series
Line
Marketing Status
Price
Package
Core
Coprocessor
MEMORY
Flash = 64 (kBytes)
64
EEPROM = 0 (Bytes)
0
RAM Total = 20 (kBytes)
20
Featu...
Block Diagr...
Docs & Resour...
CAD Resour...
Datash...
Start Proj...
STM32 Sidekick available
AI-powered tool to find and summarize STM32 technical information.
On ST Community and STM32 Developer Zone.
MCUs/MPUs List: 2 items
2
Export
Unit Price for ...
Package
Flash
RAM
I/O
Frequ...
STM32F103C...
STM32F10...
STM32F103...
Active
2.8462
LQFP 48 7x7x1.4 mm
64 kBy...
20 kBy...
37
72 MHz

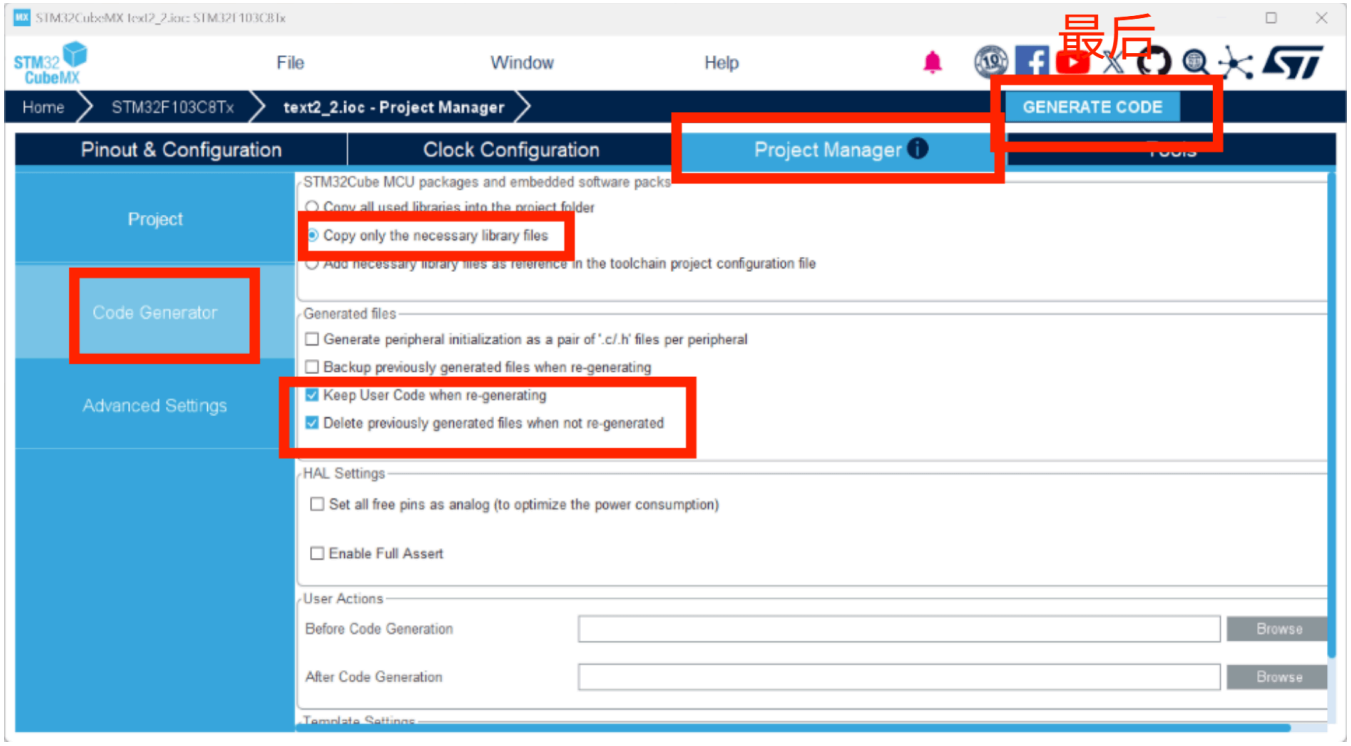

STM32CubeMX Untitled: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
System Core
Analog
Timers
1
Connectivity
CAN
I2C1
I2C2
SPI1
2
USART1
USART3
USB
Computing
Middleware and Software Packs
USART1 Mode and Configuration
Mode
Mode Disable
Disable
Asynchronous
3
Synchronous
Single Wire (Half-Duplex)
Multiprocessor Communication
IrDA
LIN
SmartCard
Configuration
Pinout view
System view
STM32F103C8Tx
LQFP48


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
Pinout & Configuration
Clock Configuration
Project Mana
Software Packs
Pinout
System Core
DMA
GPIO
IWDG
NVIC
RCC
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
RCC Mode and Configuration
Mode
High Speed Clock (HSE)
Disable
Low Speed Clock (LSE)
Disable
Master Clock Output
Crystal/Ceramic Resonator
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
Configure the below parameters :
Search (Ctrl+F)
System Parameters
VDD voltage (V)
3.3 V
Prefetch Buffer
Enabled
Flash Latency(WS)
0 WS (1 CPU cycle)
RCC Parameters
HSI Calibration Value
16
HSE Startup Timout Va...
100
LSE Startup Timout Val...
5000


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
SYS Mode and Configuration
Mode
Debug
Serial Wire
System Wake-Up
Timebase Source
SysTick
Configuration
Warning: This peripheral has no parameters to be configured.
Pinout view
System view
SYS_JTCK-SWC
SYS_JTMS-SWDIO
RCC_OSC_IN
RCC_OSC_OUT
STM32F103C8Tx
LQFP48


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Project Manager
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Project
Code Generator
Advanced Settings
Project Settings
Project Name
usart1_test
Project Location
D:\work\
Browse
Application Structure
Advanced
Do not generate the main()
Toolchain Folder Location
D:\work\usart1_test\
Toolchain / IDE
MDK-ARM
Min Version
V5.32
Generate Under Root
Linker Settings
Minimum Heap Size
0x200
Minimum Stack Size
0x400
Thread-safe Settings
Cortex-M3NS
Enable multi-threaded support
Thread-safe Locking Strategy
Default – Mapping suitable strategy depending on RTOS selection.
Mcu and Firmware Package
Mcu Reference
STM32F103C8Tx
Firmware Package Name and Version
STM32Cube FW_F1 V1.8.7
Use latest available version
Use Default Firmware Location


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Clock Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Resolve Clock Issues
LSI RC
40
40 KHz
To IWDG (KHz)
To FLITFCLK (MHz)
HSI RC
8
8 MHz
System Clock Mux
HSI
HSE
PLLCLK
SYSCLK (MHz)
AHB Prescaler
HCLK (MHz)
72 MHz max
APB1 Prescaler
PCLK1
36 MHz max
APB1 peripheral clocks (MHz)
APB1 Timer clocks (MHz)
Enable CSS
APB2 Prescaler
PCLK2
72 MHz max
APB2 peripheral clocks (MHz)
APB2 timer clocks (MHz)
ADC Prescaler
To ADC1,2
HCLK to AHB bus, core, memory and DMA (MHz)
To Cortex System timer (MHz)
FCLK (MHz)
PLL Source Mux
USB Prescaler
To USB (MHz)
Input frequency
4-16 MHz
PLL



(2) 代码编写

Includes:

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
```

PV:

```
/* USER CODE BEGIN PV */
uint8_t rx_data;
uint8_t send_flag = 1;
uint8_t tx_buffer[] = "hello windows!\r\n";
/* USER CODE END PV */
```

USER CODE BEGIN 2(在 `MX_USART1_UART_Init()` 之后):

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1, &rx_data, 1);
/* USER CODE END 2 */
```

while(1) 循环:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if (send_flag == 1)
    {
        HAL_UART_Transmit(&huart1, tx_buffer, strlen((char*)tx_buffer), 1000);
        HAL_Delay(500); // 必须加延时，否则上位机接收缓冲区溢出丢包严重
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

USER CODE BEGIN 4:

```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (rx_data == '#')
        {
            send_flag = 0; // 收到#，暂停发送
        }
        else if (rx_data == '*')
        {
            send_flag = 1; // 收到*，恢复发送
        }
        // 重新开启接收中断
        HAL_UART_Receive_IT(&huart1, &rx_data, 1);
    }
}
/* USER CODE END 4 */
```

完整活动

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
```

```
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>  // 需要用到 strlen 函数
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
uint8_t rx_data;             // 接收单字节的缓存
uint8_t send_flag = 1;       // 发送标志位：1为发送，0为暂停
uint8_t tx_buffer[] = "hello windows!\r\n"; // 要发送的字符串
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
```

```
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  // 初始化完成后，立刻开启第一次接收中断，等待上位机发送字符
  HAL_UART_Receive_IT(&huart1, &rx_data, 1);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
      if (send_flag == 1)
      {
          // 发送字符串
          HAL_UART_Transmit(&huart1, tx_buffer, strlen((char*)tx_buffer), 1000);

          // 这里加了 500ms 延时。因为如果连续无延时发送，上位机（串口助手）处理不过来会导致严重丢包甚至软
件卡死。
          // 如果想观察任务一中提到的丢包现象，可以将下面这行延时注释掉再测试。
          HAL_Delay(500);
      }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
```

```c
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief USART1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART1_UART_Init(void)
{

  /* USER CODE BEGIN USART1_Init 0 */

  /* USER CODE END USART1_Init 0 */

  /* USER CODE BEGIN USART1_Init 1 */

  /* USER CODE END USART1_Init 1 */
```

```
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART1_Init 2 */

  /* USER CODE END USART1_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

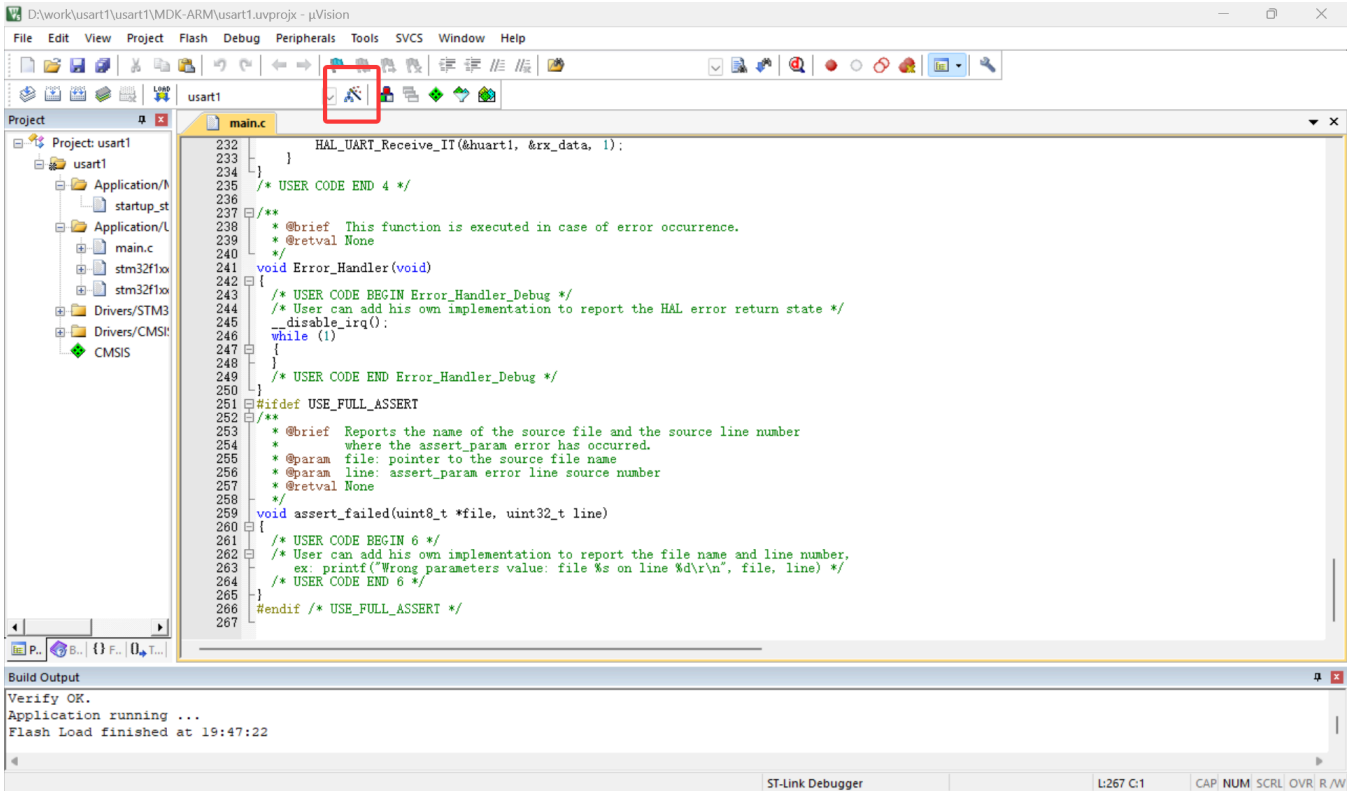
/* USER CODE BEGIN 4 */
/**
  * @brief  Rx Transfer completed callback.
  * @param  huart UART handle.
  * @retval None
  */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // 判断是否是 USART1 触发的中断
    if (huart->Instance == USART1)
    {
        if (rx_data == '#')
        {
            send_flag = 0;  // 收到 '#'，暂停发送
        }
```

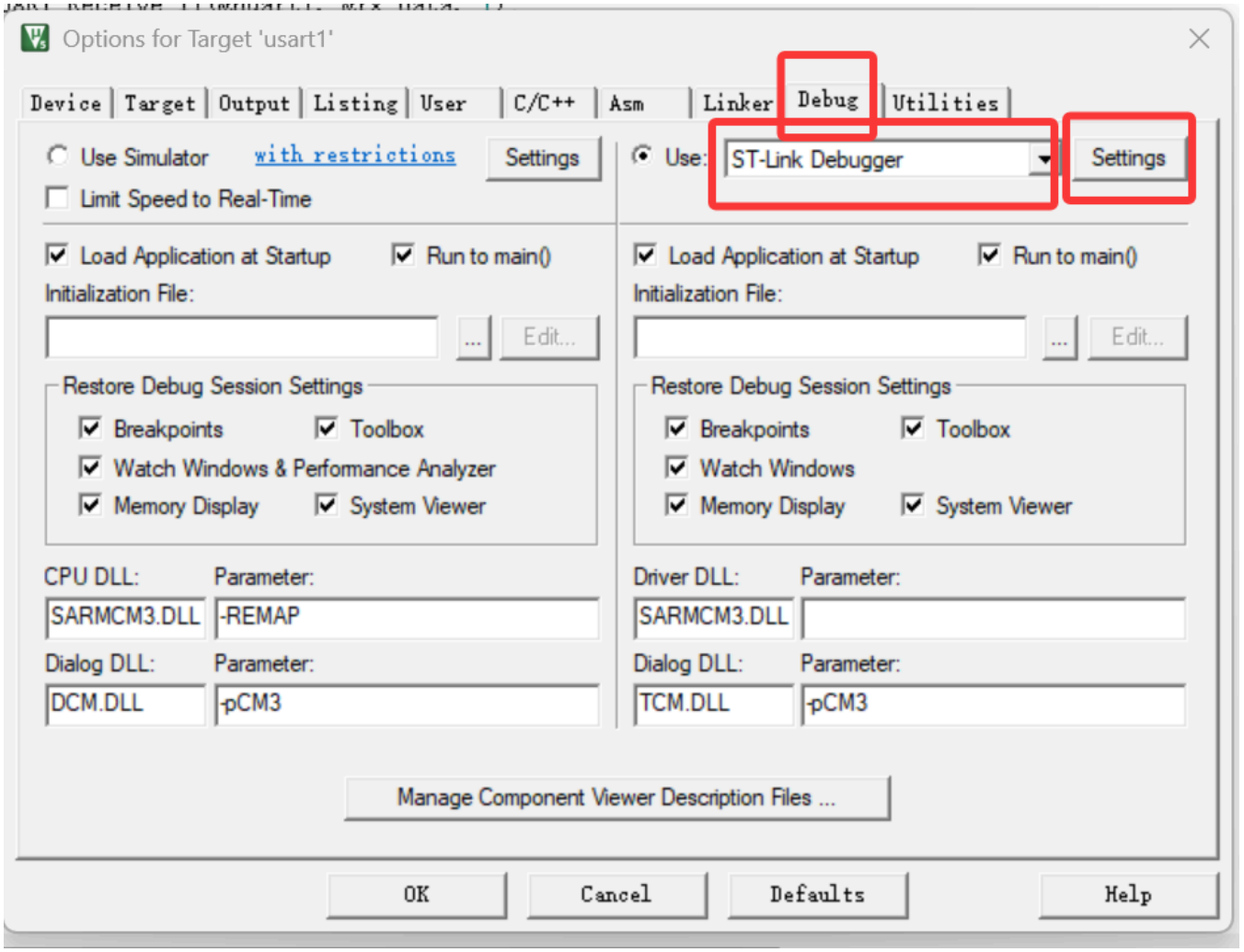
```
        else if (rx_data == '*')
        {
            send_flag = 1;  // 收到 '*'，恢复发送
        }

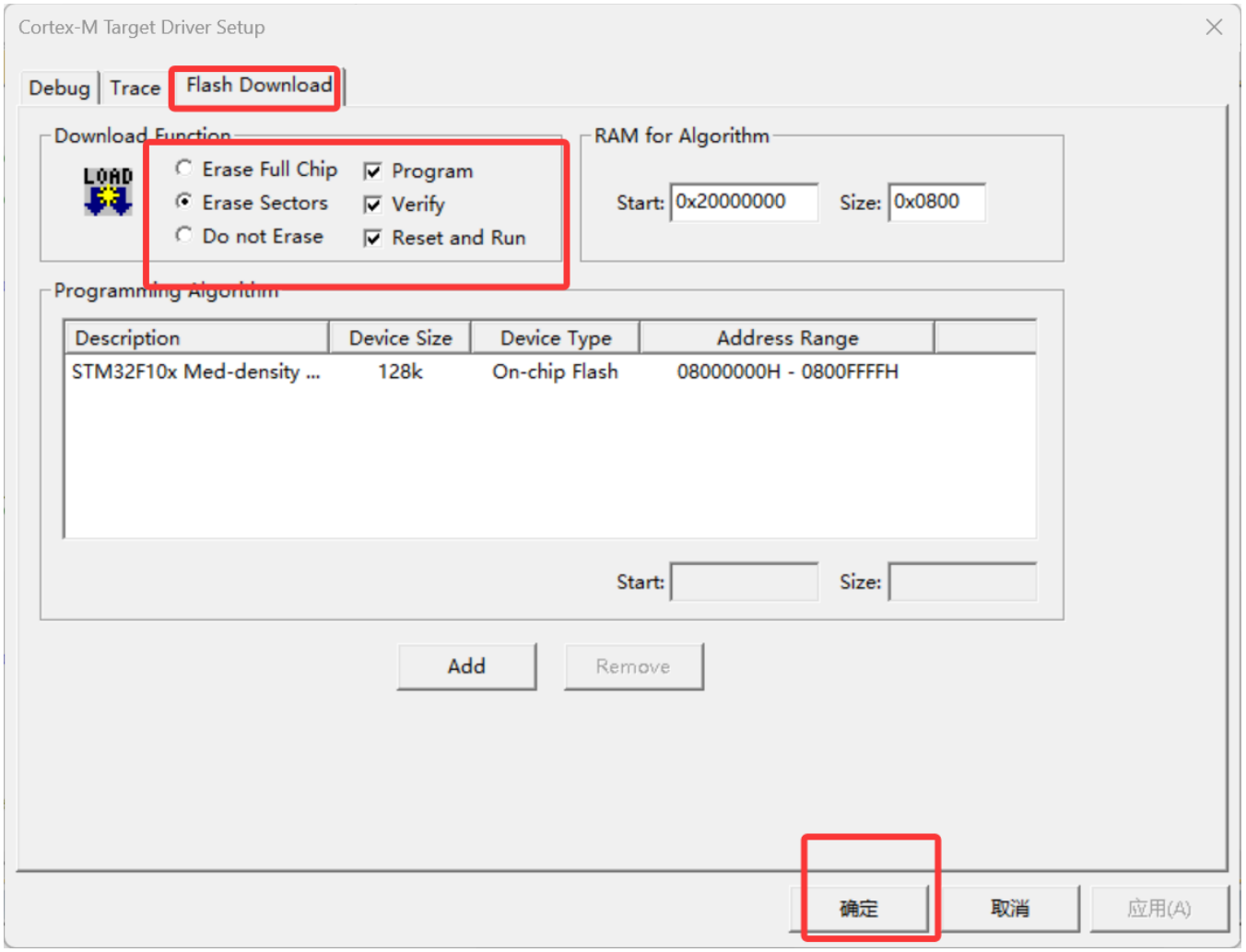
        // 处理完后，必须重新开启接收中断，以便接收下一个字符
        HAL_UART_Receive_IT(&huart1, &rx_data, 1);
    }
}
/* USER CODE END 4 */

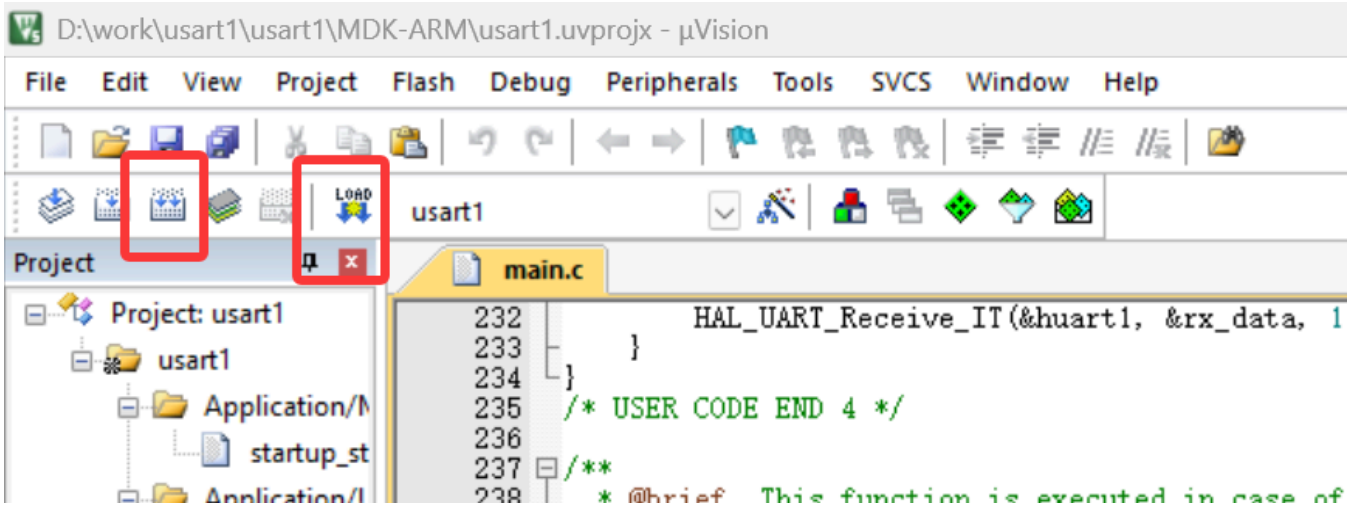
/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

3.代码编译及下载


D:\work\usart1\usart1\MDK-ARM\usart1.uvprojx - µVision
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
usart1
Project
Project: usart1
usart1
Application/M
startup_st
Application/U
main.c
stm32f1xx
stm32f1xx
Drivers/STM3
Drivers/CMSI
CMSIS
main.c
HAL_UART_Receive_IT(&huart1, &rx_data, 1);
}
}
/* USER CODE END 4 */
/**
* @brief This function is executed in case of error occurrence.
* @retval None
*/
void Error_Handler(void)
{
/* USER CODE BEGIN Error_Handler_Debug */
/* User can add his own implementation to report the HAL error return state */
__disable_irq();
while (1)
{
}
/* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
* @brief Reports the name of the source file and the source line number
* where the assert_param error has occurred.
* @param file: pointer to the source file name
* @param line: assert_param error line source number
* @retval None
*/
void assert_failed(uint8_t *file, uint32_t line)
{
/* USER CODE BEGIN 6 */
/* User can add his own implementation to report the file name and line number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
/* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
Build Output
Verify OK.
Application running ...
Flash Load finished at 19:47:22
ST-Link Debugger
L:267 C:1
CAP NUM SCRL OVR R/W


Options for Target 'usart1'
Device Target Output Listing User C/C++ Asm Linker Debug Utilities
Use Simulator with restrictions Settings
Limit Speed to Real-Time
Use: ST-Link Debugger Settings
Load Application at Startup Run to main()
Initialization File:
Edit...
Restore Debug Session Settings
Breakpoints Toolbox
Watch Windows & Performance Analyzer
Memory Display System Viewer
Load Application at Startup Run to main()
Initialization File:
Edit...
Restore Debug Session Settings
Breakpoints Toolbox
Watch Windows
Memory Display System Viewer
CPU DLL: Parameter:
SARMCM3.DLL -REMAP
Dialog DLL: Parameter:
DCM.DLL -pCM3
Driver DLL: Parameter:
SARMCM3.DLL
Dialog DLL: Parameter:
TCM.DLL -pCM3
Manage Component Viewer Description Files ...
OK Cancel Defaults Help





4.实现串口通信

打开sscom


SSCOM V5.13.1 串口/网络数据调试器,作者:大虾丁丁,2618058@qq.com. QQ群: 52502449(最新版本)
通讯端口 串口设置 显示 发送 多字符串 小工具 帮助 联系作者 大虾电子网
清除窗口
打开文件
发送文件
停止
清发送区
最前
English
保存参数
扩展
端口号 COM6 USB-SERIAL CH340
HEX显示
保存数据
接收数据到文件
HEX发送
定时发送: 1000 ms/次
加回车换行
打开串口
更多串口设置
加时间戳和分包显示,
超时时间: 20 ms
第 1 字节 至 末尾
加校验 None
RTS DTR 波特率: 115200
为了更好地发展SSCOM软件
请您注册嘉立创P结尾客户
发 送
▲Q群满员了,没有第二个群. ★合宙高性价比4G模块 ★RT-Thread中国人的开源免费操作系统 ★ ★8KM远距离WiFi可自组网
www.daxia.com
S:0
R:0
COM6 已关闭 115200bps,8,1,None,None

四、任务二

1.实验目的

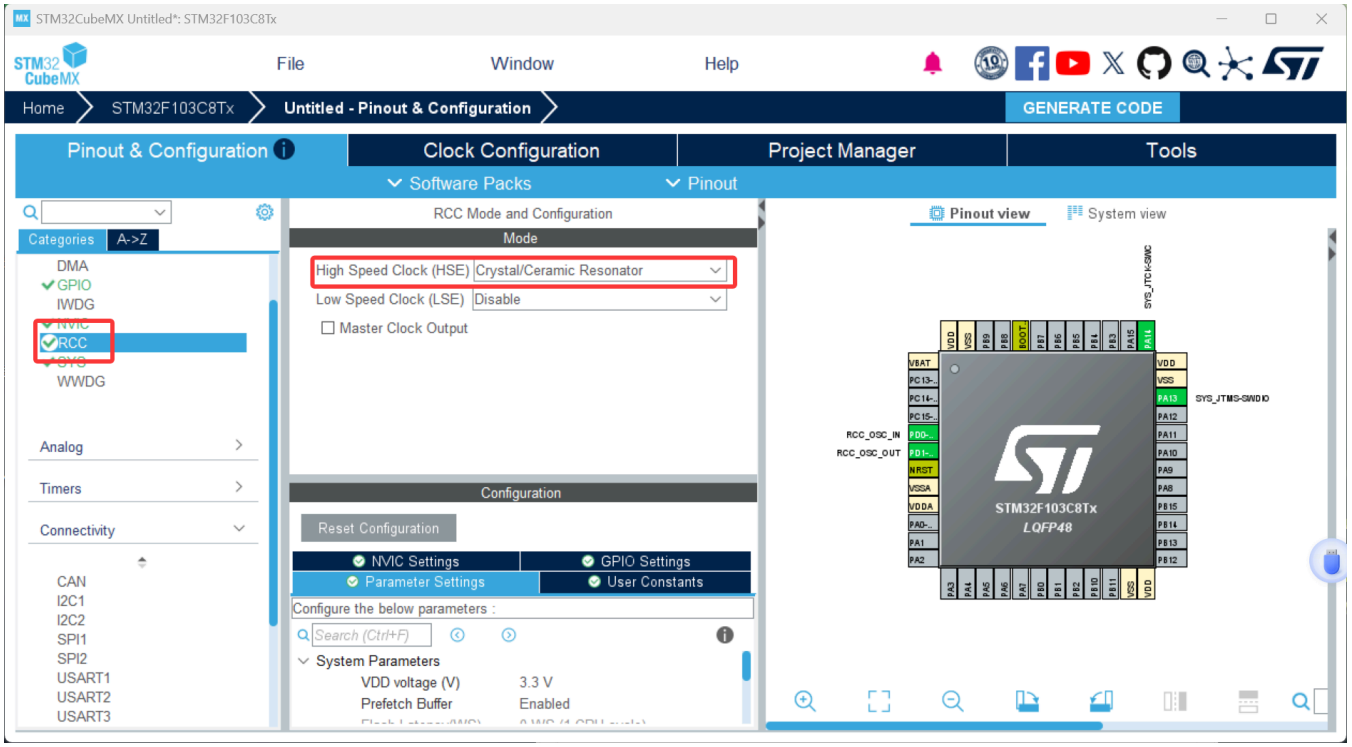
学习STM32中断、DMA通信原理和编程方法。使用stm32tubemx和HAL库分别完成以下编程练习：

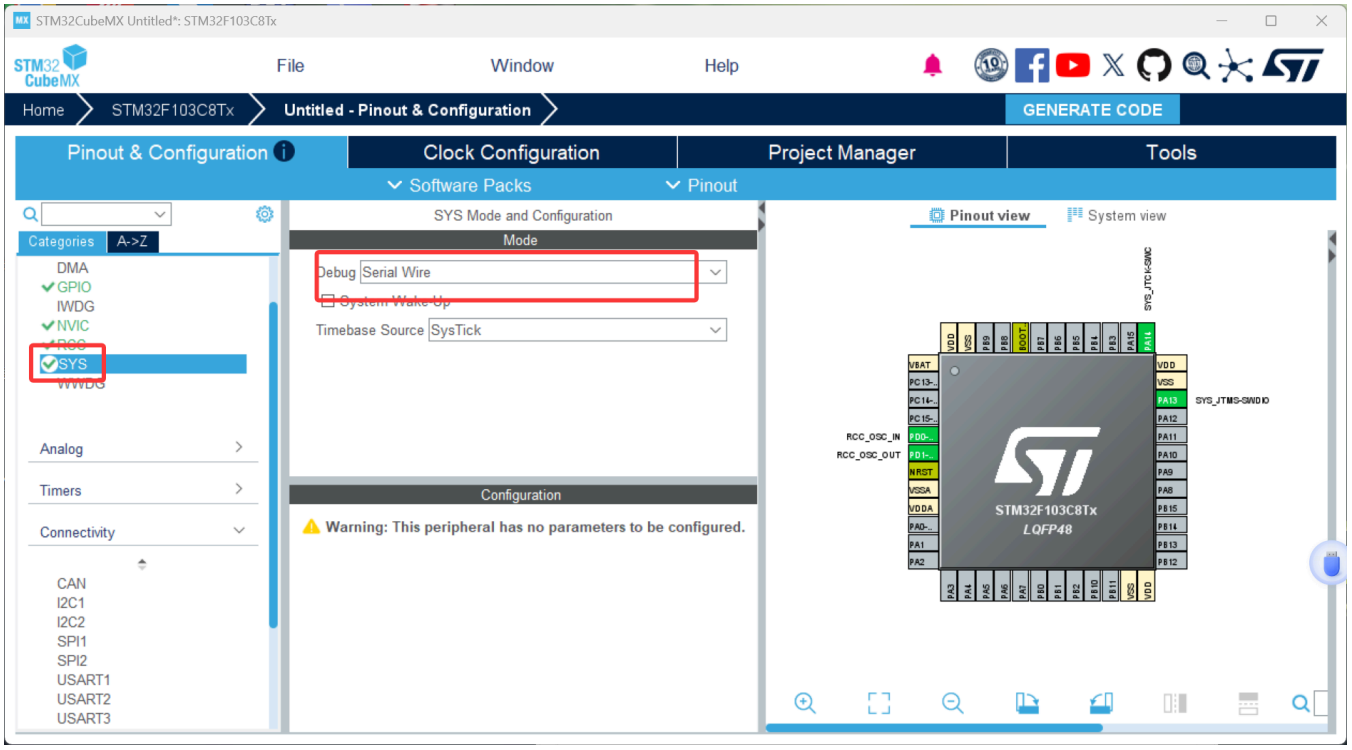
（1） 采用串口中断方式重做上面任务一（2）的串口通信实验。

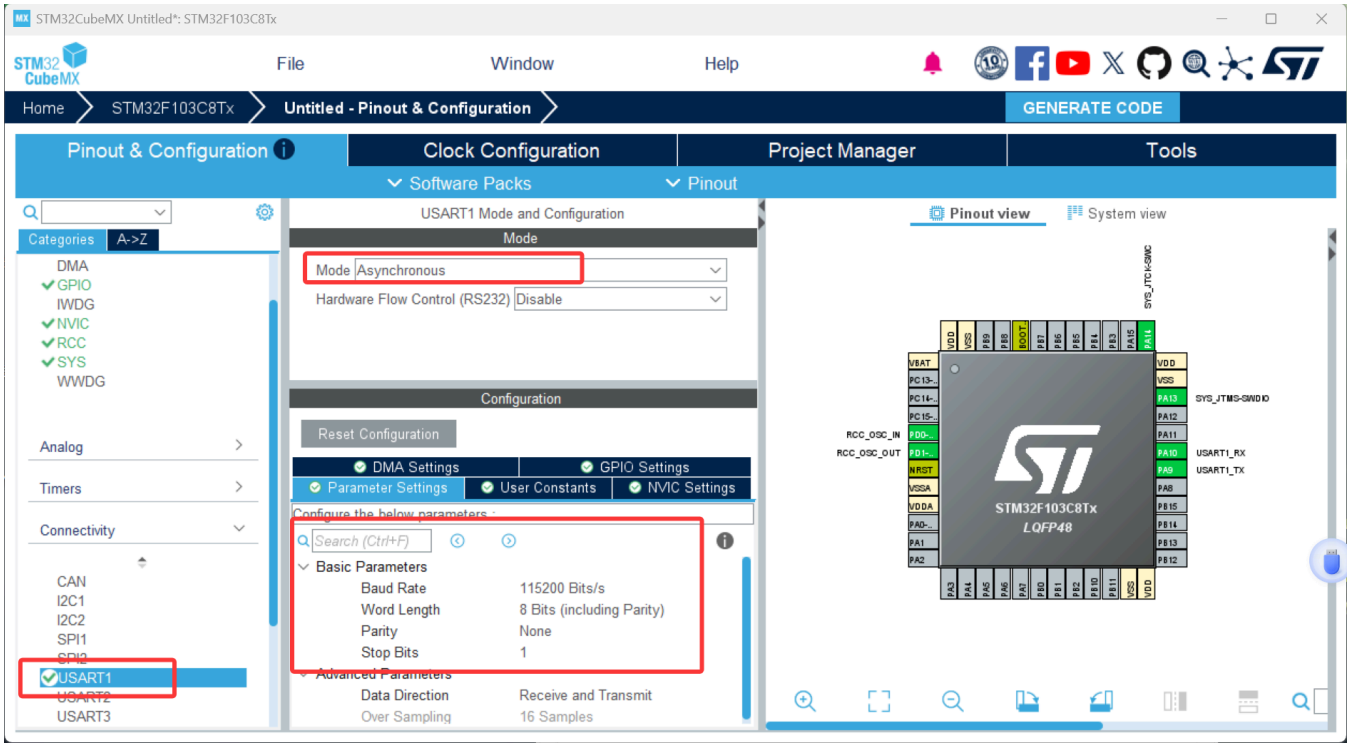
（2）. STM32采用串口DMA方式，用115200bps或更高速率向上位机连续发送数据。

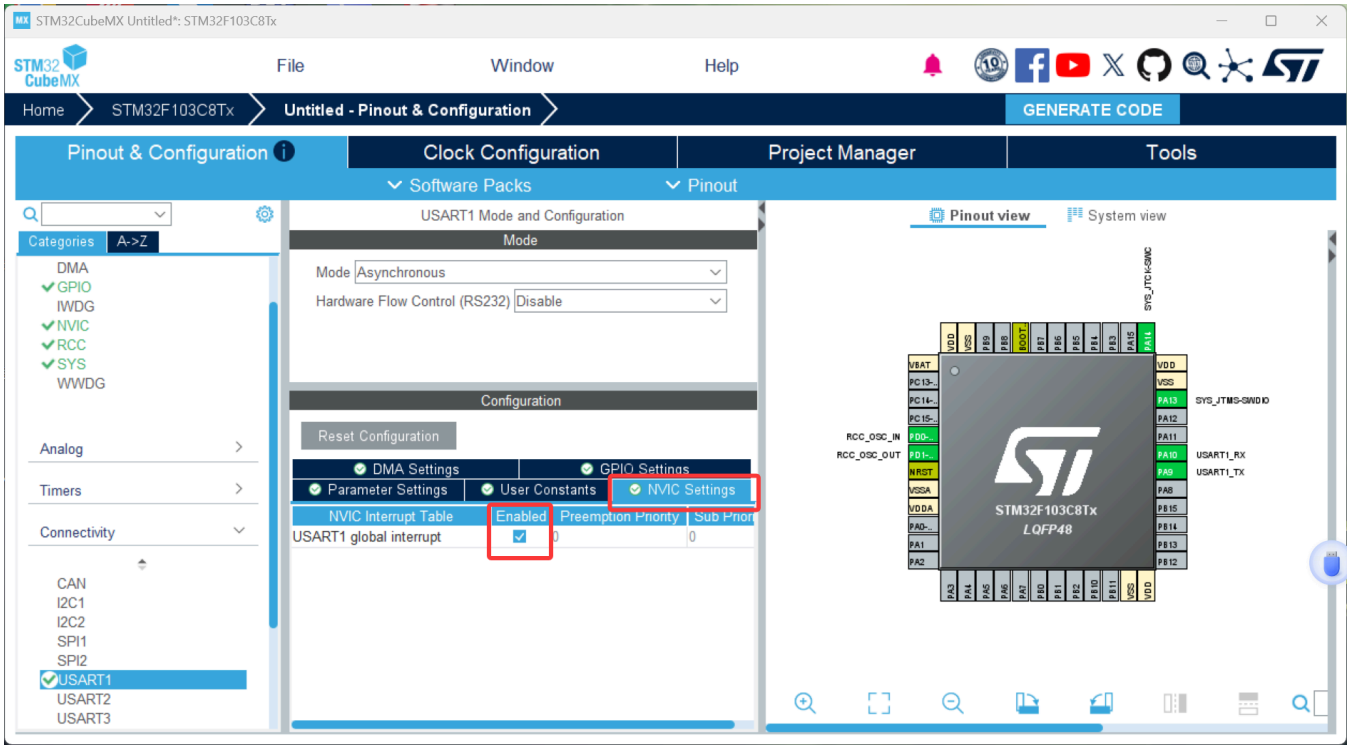
2.串口中断工程实现

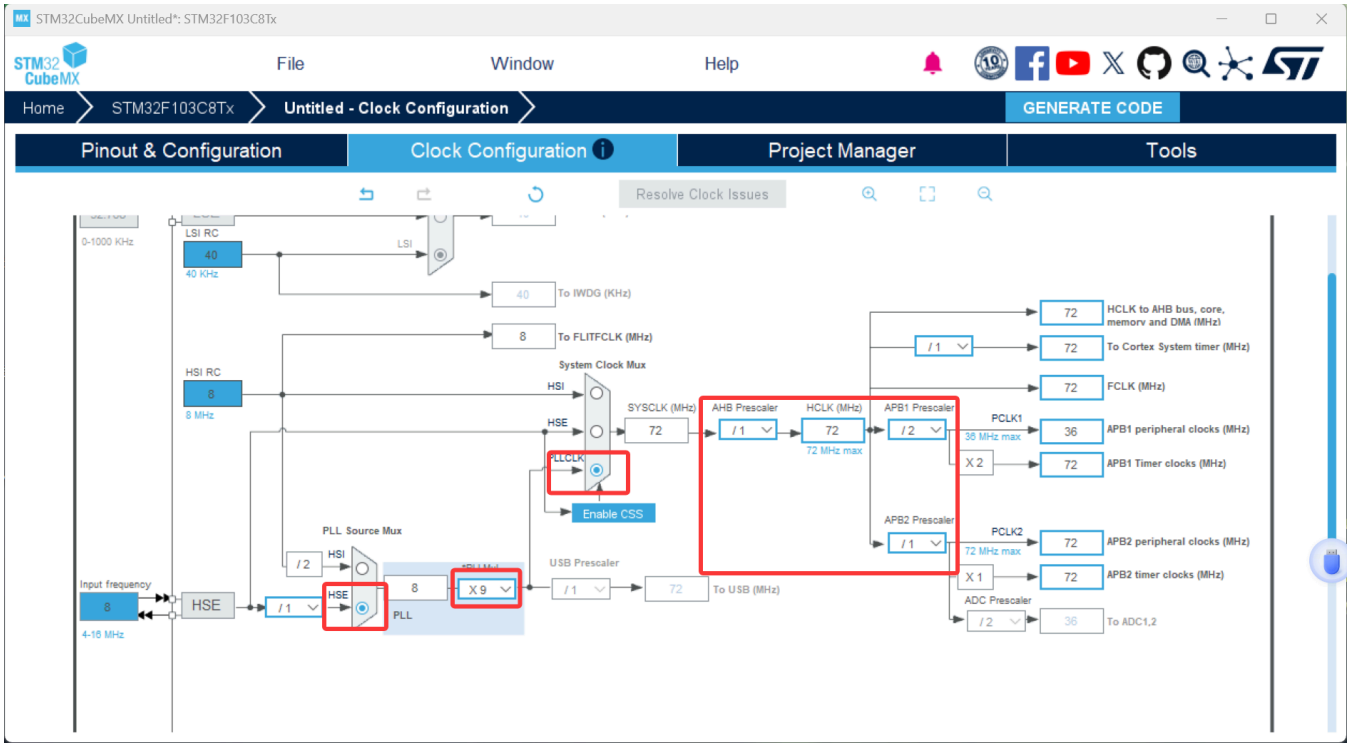
（1）创建工程

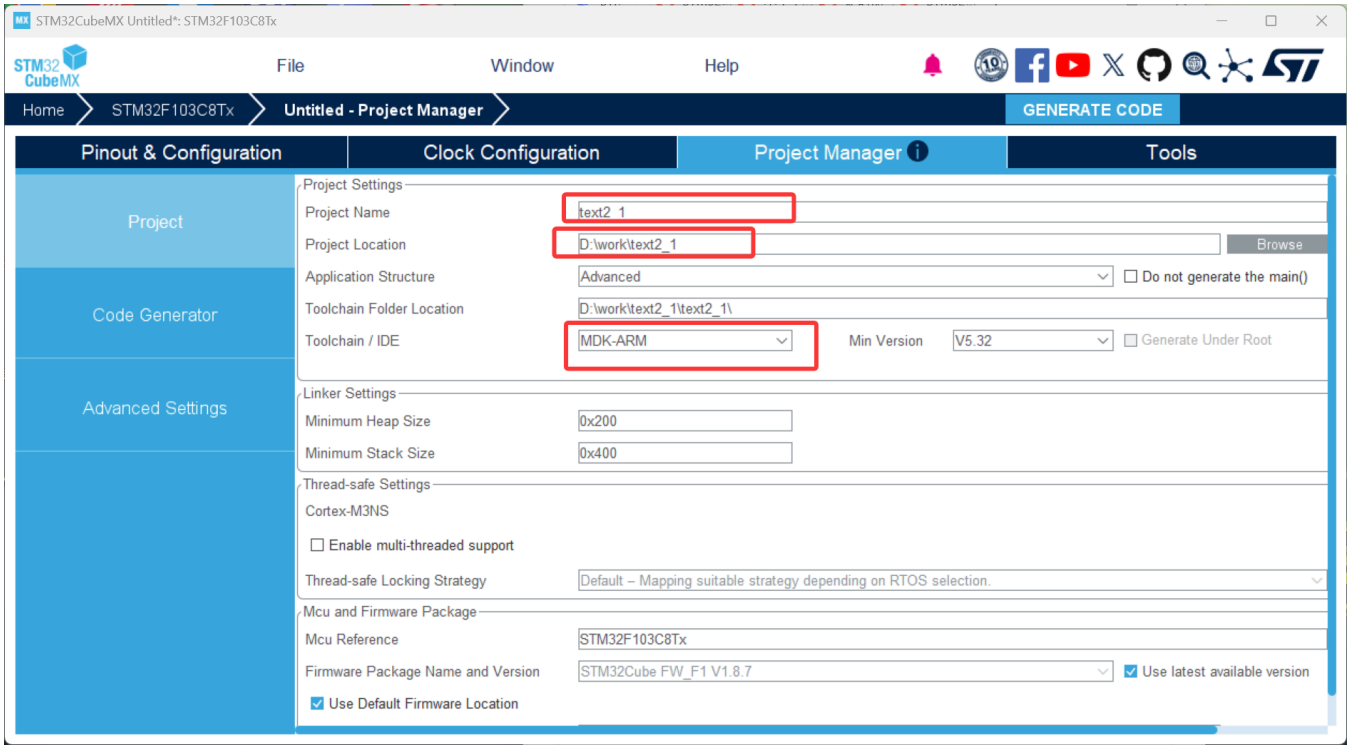

STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
RCC Mode and Configuration
Mode
High Speed Clock (HSE)
Crystal/Ceramic Resonator
Low Speed Clock (LSE)
Disable
Master Clock Output
Configuration
Reset Configuration
NVIC Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
System Parameters
VDD voltage (V)
3.3 V
Prefetch Buffer
Enabled
Pinout view
System view
SYS_JTCK-SWC
SYS_JTMS-SWDIO
RCC_OSC_IN
RCC_OSC_OUT
STM32F103C8Tx
LQFP48


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
SYS Mode and Configuration
Mode
Debug
Serial Wire
Timebase Source
SysTick
Configuration
Warning: This peripheral has no parameters to be configured.
Pinout view
System view
SYS_JTCK-SWC
SYS_JTMS-SWDIO
RCC_OSC_IN
RCC_OSC_OUT
STM32F103C8Tx
LQFP48


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Hardware Flow Control (RS232) Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
Search (Ctrl+F)
Basic Parameters
Baud Rate 115200 Bits/s
Word Length 8 Bits (including Parity)
Parity None
Stop Bits 1
Advanced Parameters
Data Direction Receive and Transmit
Over Sampling 16 Samples
Pinout view
System view
STM32F103C8Tx
LQFP48
SYS_JTMS-SWDIO
USART1_RX
USART1_TX
RCC_OSC_IN
RCC_OSC_OUT


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Hardware Flow Control (RS232) Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
NVIC Interrupt Table
Enabled
Preemption Priority
Sub Prior
USART1 global interrupt
0
0
Pinout view
System view
STM32F103C8Tx
LQFP48
SYS_JTMS-SWDIO
USART1_RX
USART1_TX
RCC_OSC_IN
RCC_OSC_OUT





(2) 编写代码

在`main.c`中编写以下代码：

```
/* USER CODE BEGIN PV */
uint8_t rx_data;             // 接收单字节缓存
uint8_t send_flag = 1;       // 1为发送，0为暂停
uint8_t tx_buffer[] = "hello windows!\r\n";
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1, &rx_data, 1); // 开启第一次接收中断
```

```
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    if (send_flag == 1)
    {
        HAL_UART_Transmit(&huart1, tx_buffer, strlen((char*)tx_buffer), 1000);
        HAL_Delay(500); // 建议加入延时，避免上位机丢包
    }
    /* USER CODE END WHILE */
}
/* USER CODE END 3 */

/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (rx_data == '#')
        {
            send_flag = 0;  // 收到'#'，暂停发送
        }
        else if (rx_data == '*')
        {
            send_flag = 1;  // 收到'*'，恢复发送
        }
        HAL_UART_Receive_IT(&huart1, &rx_data, 1); // 重新开启中断
    }
}
/* USER CODE END 4 */
```

完整代码

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>  // 需要用到 strlen() 计算字符串长度
/* USER CODE END Includes */
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
```

```c
/* USER CODE END PD */
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ---------------------------------------------------------*/
UART_HandleTypeDef huart1;
/* USER CODE BEGIN PV */
volatile uint8_t rx_data;                    // 仅这里增加volatile，其余不变
uint8_t send_flag = 1;              // 发送标志位：1表示允许发送，0表示暂停发送
volatile uint8_t tx_done_flag = 1;  // 发送完成标志位：1表示上一帧已发完，0表示还在发（防止HAL_BUSY）
uint8_t tx_buffer[] = "hello windows!\r\n"; // 要发送的字符串
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */
int main(void)
{
  /* USER CODE BEGIN 1 */
  /* USER CODE END 1 */
  /* MCU Configuration--------------------------------------------------------*/
  HAL_Init();
  SystemClock_Config();
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART1_UART_Init();

  /* USER CODE BEGIN 2 */
  // 初始化完成后，立刻开启接收中断，准备接收上位机发来的字符
  HAL_UART_Receive_IT(&huart1, &rx_data, 1);
  /* USER CODE END 2 */
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    // 只有当允许发送(send_flag==1)，且上一帧已经发送完成(tx_done_flag==1)时，才启动新的一帧
    if (send_flag == 1 && tx_done_flag == 1)
    {
        tx_done_flag = 0; // 清除发送完成标志
        // 采用中断方式发送！
        HAL_UART_Transmit_IT(&huart1, tx_buffer, strlen((char*)tx_buffer));
    }

    // 延时500ms，控制发送频率。如果不用延时，CPU会疯狂重复发起中断，导致发送混乱。
    HAL_Delay(500);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
```

```c
  }
  /* USER CODE END 3 */
}
/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  // （此处省略系统时钟配置，维持你之前的72MHz配置）
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) Error_Handler();
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK) Error_Handler();
}
/**
  * @brief USART1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART1_UART_Init(void)
{
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK) Error_Handler();
}
/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  __HAL_RCC_GPIOD_CLK_ENABLE();
```

```
  __HAL_RCC_GPIOA_CLK_ENABLE();
}
/* USER CODE BEGIN 4 */
/**
  * @brief  串口接收完成中断回调函数（处理上位机发来的 # 和 *）
  */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (rx_data == '#')
        {
            send_flag = 0;  // 收到'#'，暂停发送
        }
        else if (rx_data == '*')
        {
            send_flag = 1;  // 收到'*'，恢复发送
        }

        // 必须重新开启接收中断，否则只能接收一次数据
        HAL_UART_Receive_IT(&huart1, &rx_data, 1);
    }
}
/**
  * @brief  串口发送完成中断回调函数（通知主循环可以发下一帧了）
  */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        tx_done_flag = 1; // 置位发送完成标志，允许主循环发起下一次发送
    }
}
/* USER CODE END 4 */
/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
  }
}
```

（3）代码编译及下载

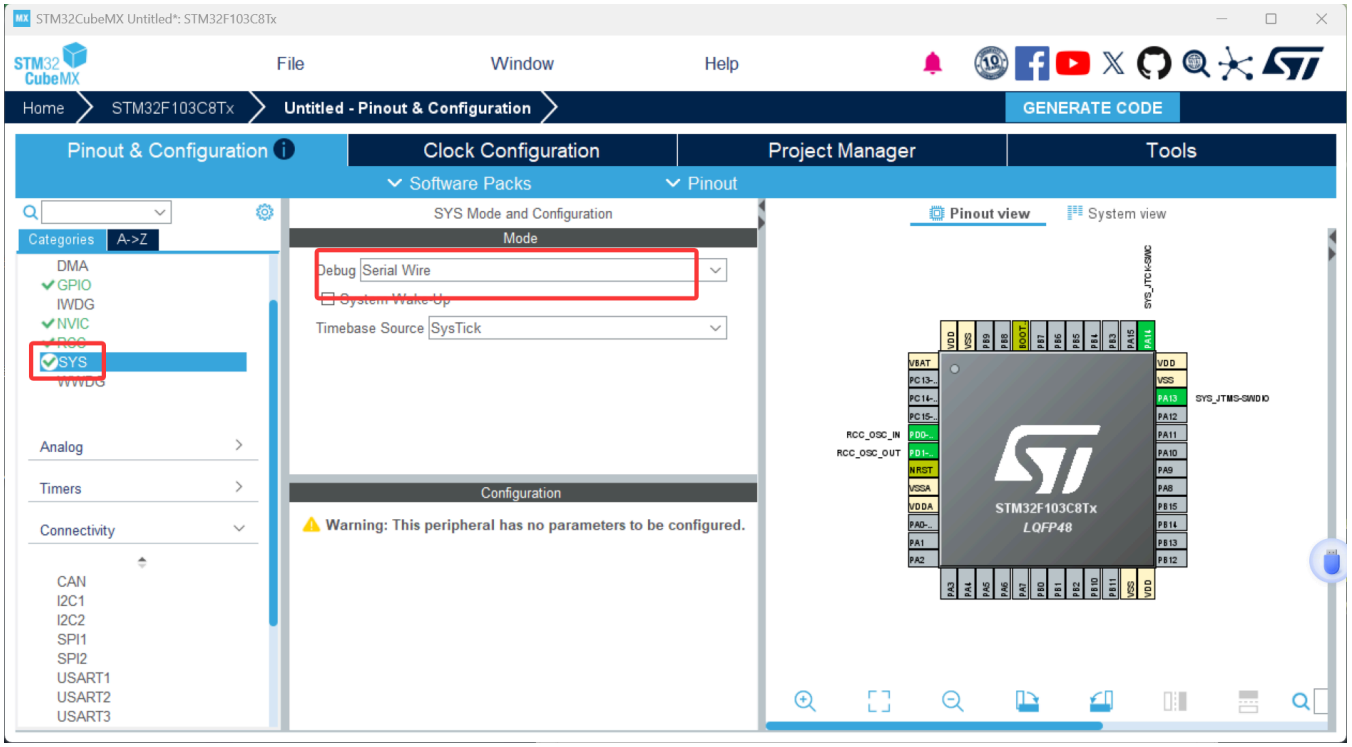
与上述过程一致

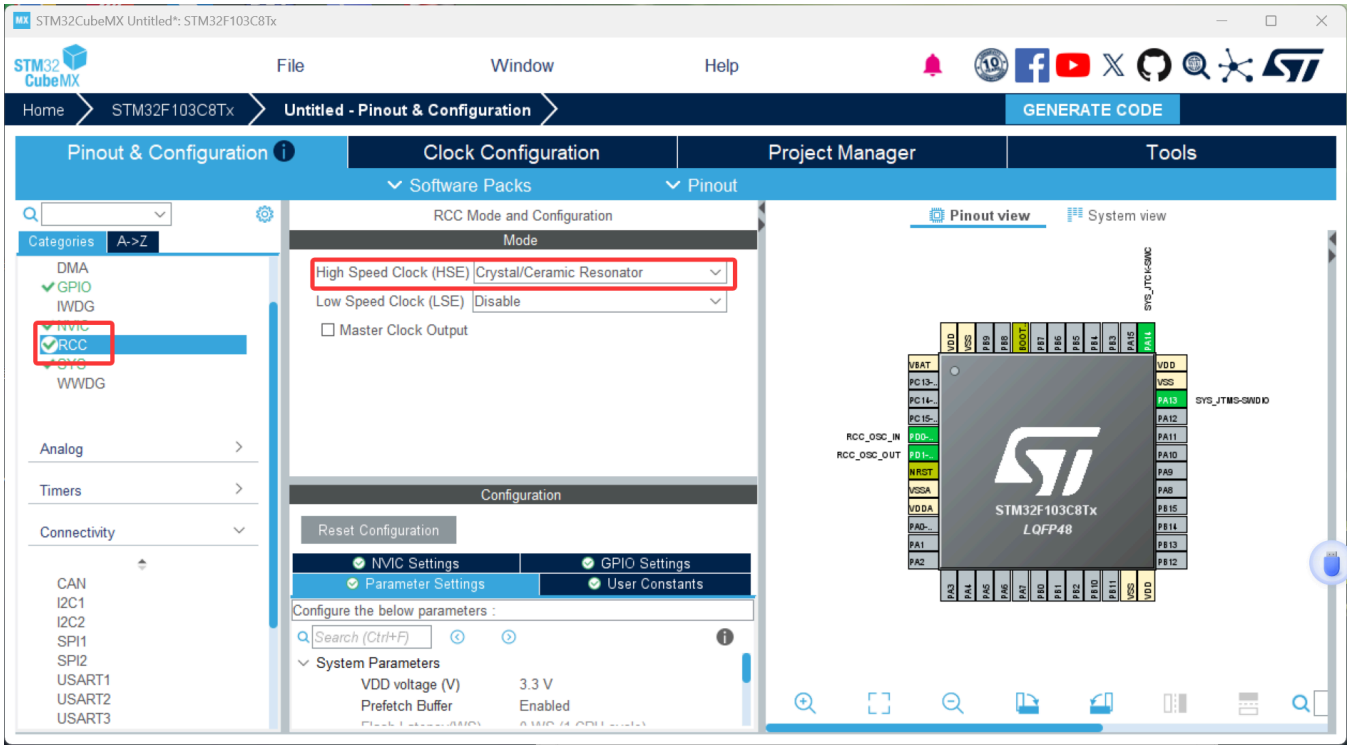
（4）调用串口助手查看实验结果

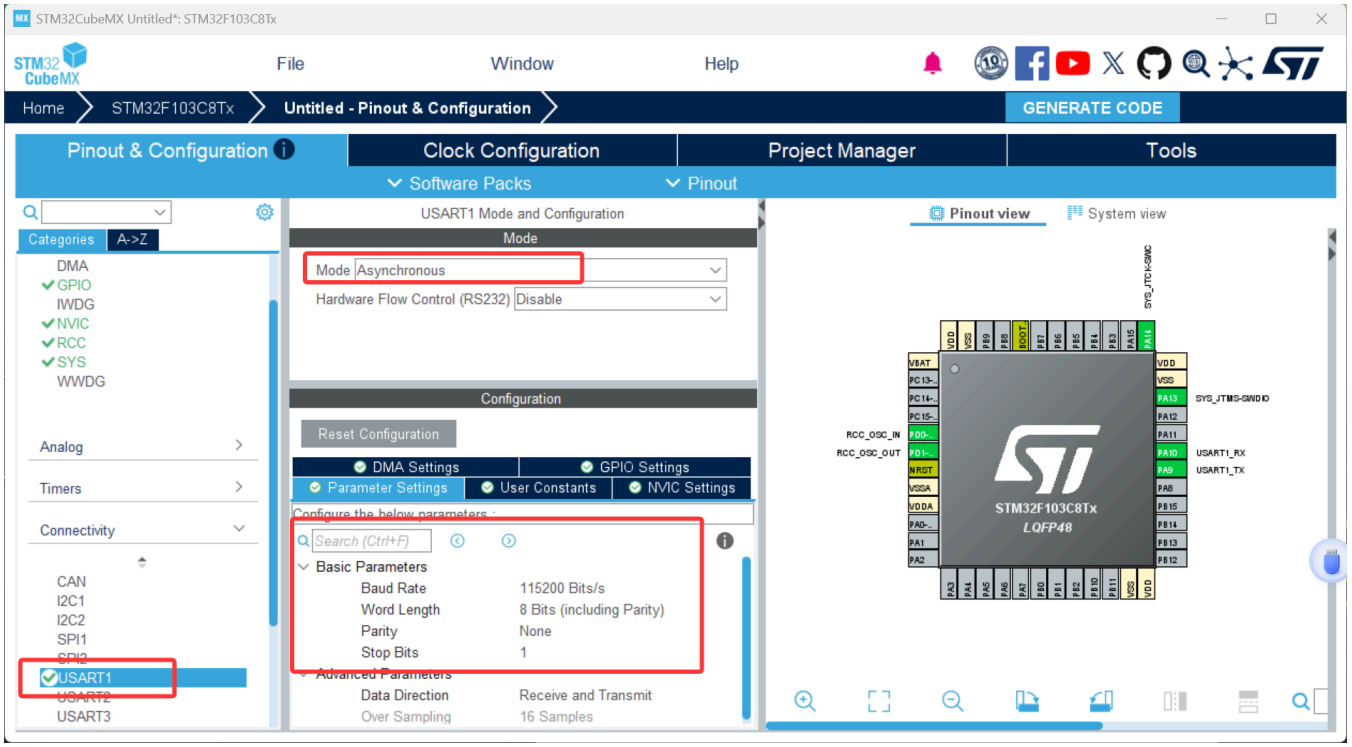


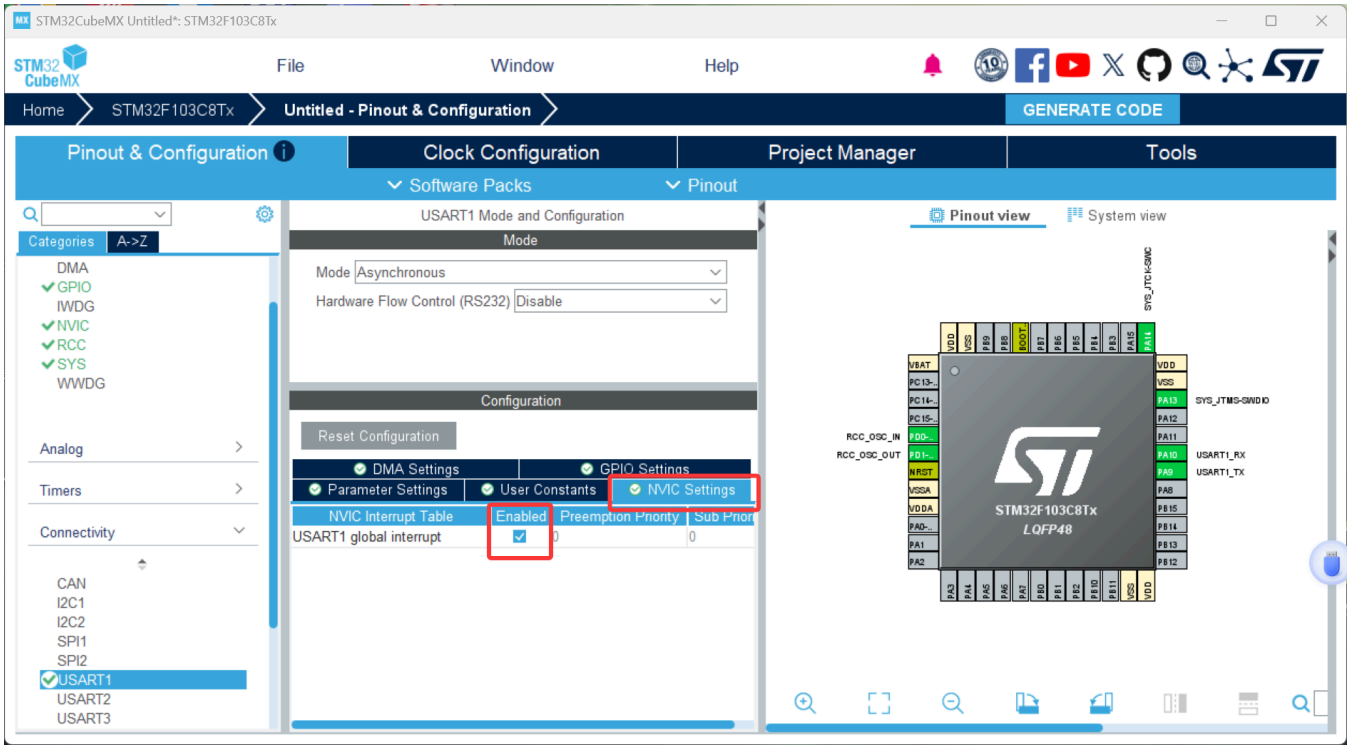
3.串口中断工程实现

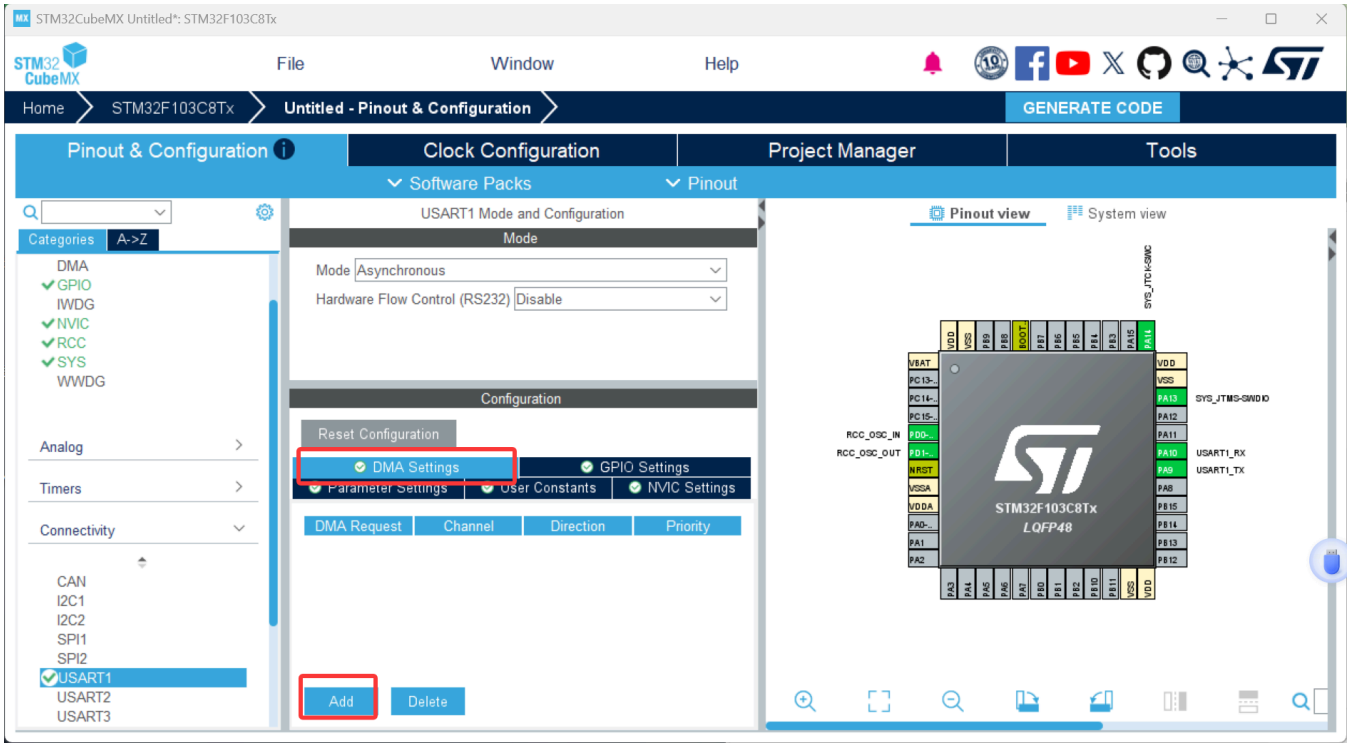
（1）创建工程

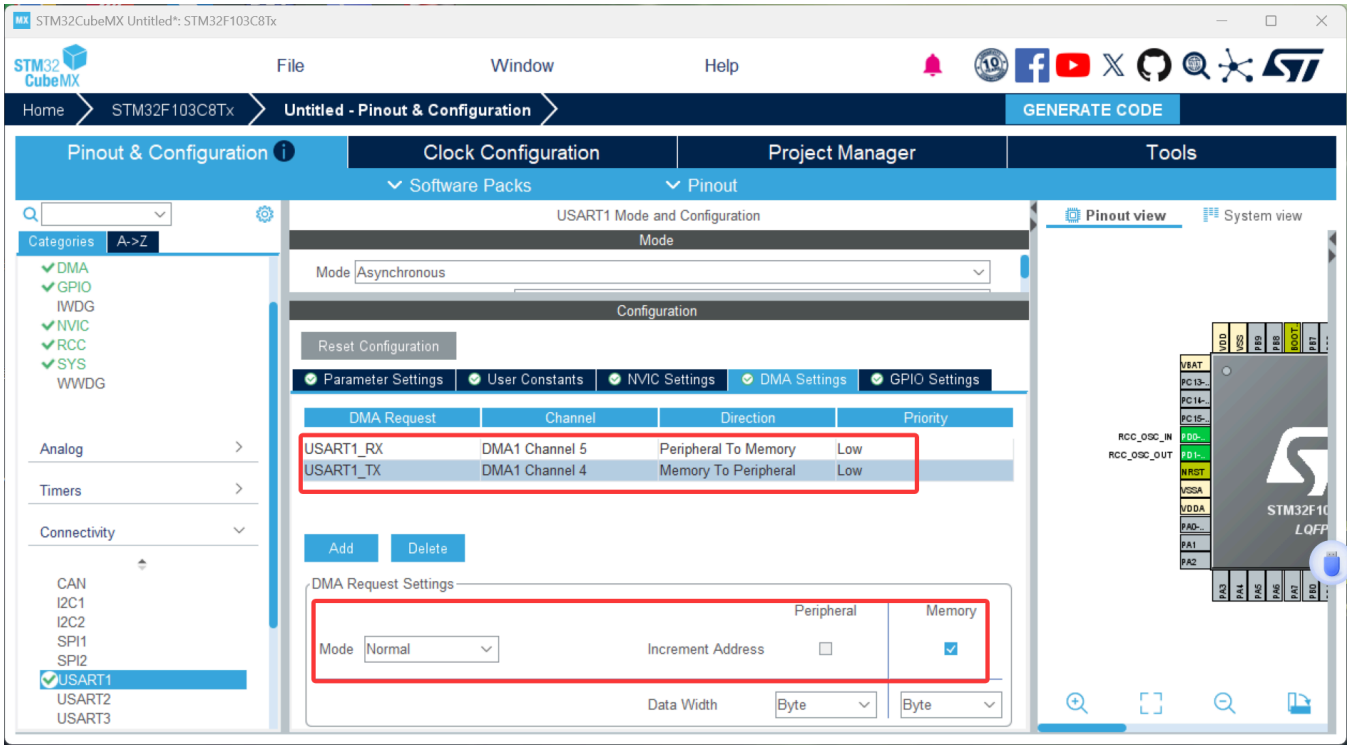

STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
SYS Mode and Configuration
Mode
Debug Serial Wire
Timebase Source SysTick
Configuration
Warning: This peripheral has no parameters to be configured.
Pinout view
System view
SYS_JTCK-SWC
SYS_JTMS-SWDIO
RCC_OSC_IN
RCC_OSC_OUT
STM32F103C8Tx
LQFP48

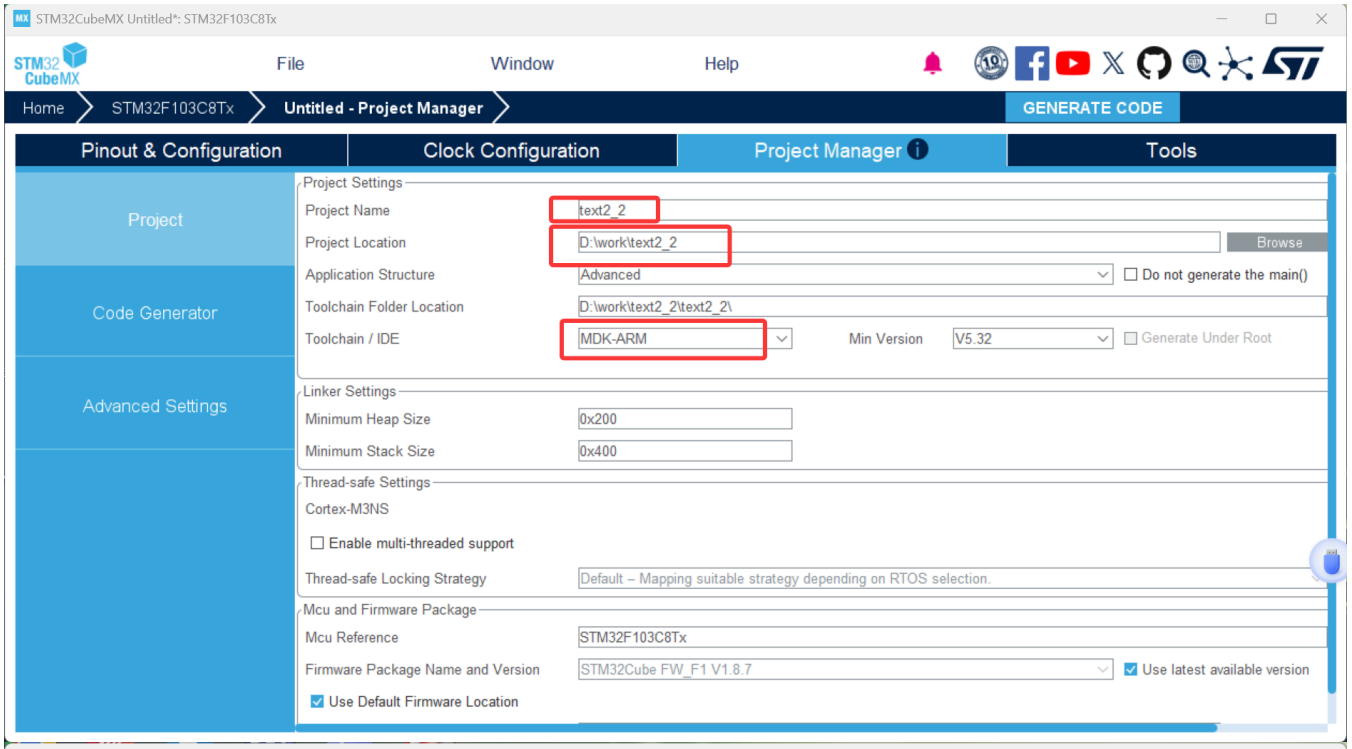

STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
RCC Mode and Configuration
Mode
High Speed Clock (HSE) Crystal/Ceramic Resonator
Low Speed Clock (LSE) Disable
Master Clock Output
Configuration
Reset Configuration
NVIC Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
System Parameters
VDD voltage (V) 3.3 V
Prefetch Buffer Enabled
Pinout view
System view
SYS_JTCK-SWC
SYS_JTMS-SWDIO
RCC_OSC_IN
RCC_OSC_OUT
STM32F103C8Tx
LQFP48

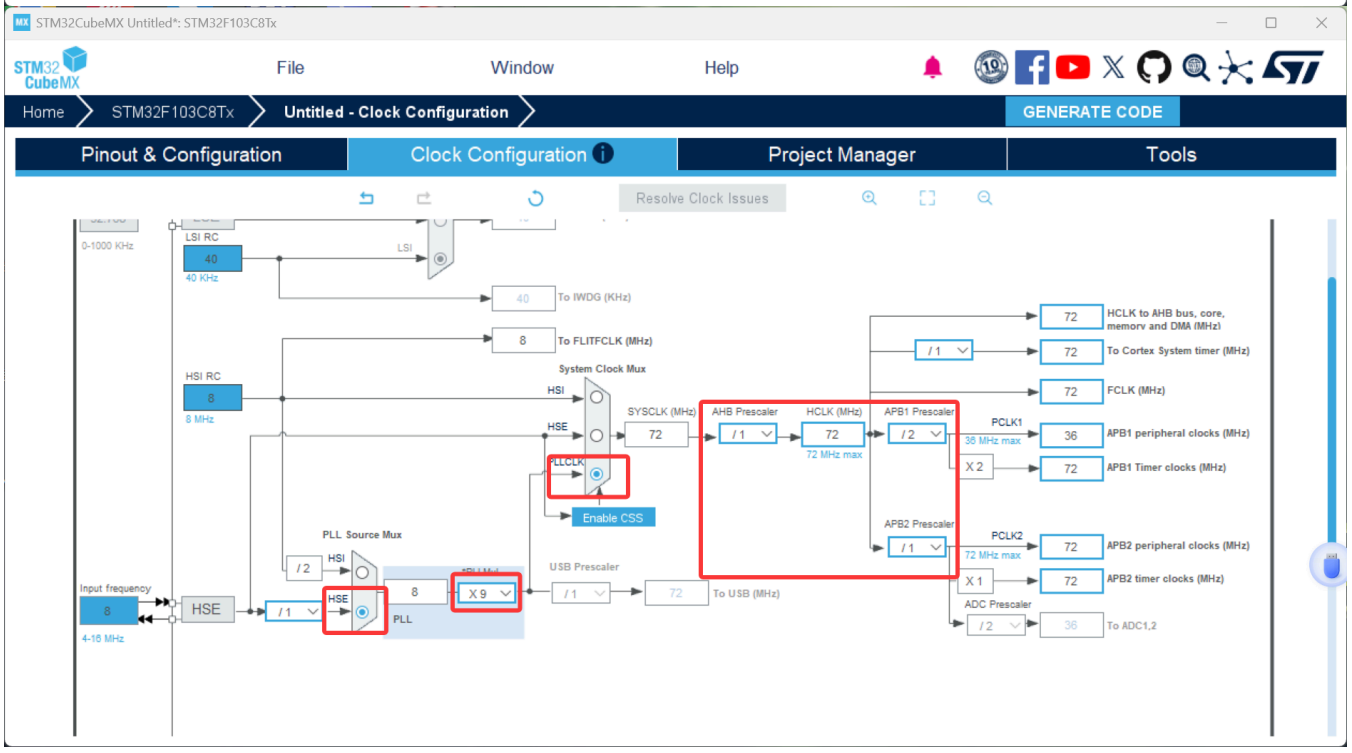

STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Hardware Flow Control (RS232) Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
Search (Ctrl+F)
Basic Parameters
Baud Rate 115200 Bits/s
Word Length 8 Bits (including Parity)
Parity None
Stop Bits 1
Advanced Parameters
Data Direction Receive and Transmit
Over Sampling 16 Samples
Pinout view
System view
STM32F103C8Tx
LQFP48
SYS_JTMS-SWDIO
USART1_RX
USART1_TX
RCC_OSC_IN
RCC_OSC_OUT

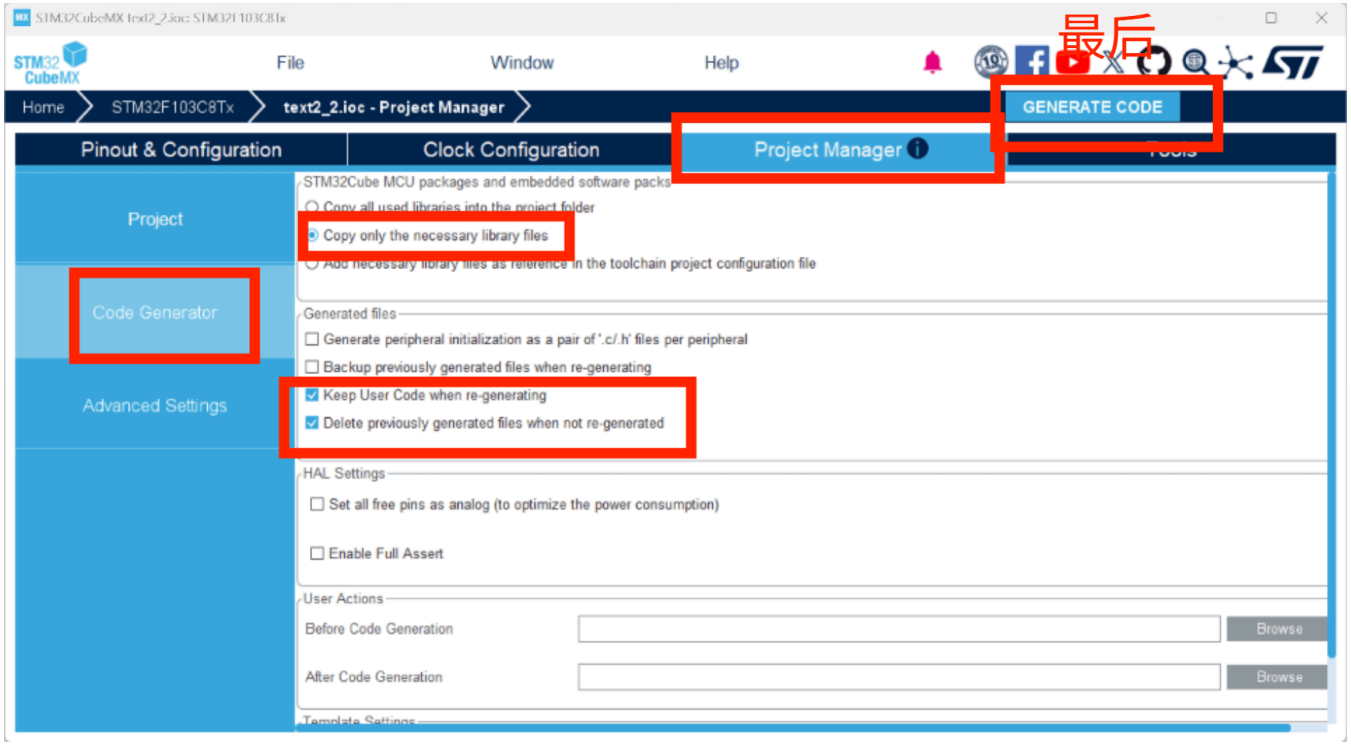

STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Hardware Flow Control (RS232) Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
NVIC Interrupt Table
Enabled
Preemption Priority
Sub Prior
USART1 global interrupt
0
0
Pinout view
System view
STM32F103C8Tx
LQFP48
SYS_JTMS-SWDIO
USART1_RX
USART1_TX
RCC_OSC_IN
RCC_OSC_OUT


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Hardware Flow Control (RS232) Disable
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
DMA Request
Channel
Direction
Priority
Add
Delete
Pinout view
System view
STM32F103C8Tx
LQFP48
SYS_JTMS-SWDIO
USART1_RX
USART1_TX
RCC_OSC_IN
RCC_OSC_OUT


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
DMA
GPIO
IWDG
NVIC
RCC
SYS
WWDG
Analog
Timers
Connectivity
CAN
I2C1
I2C2
SPI1
SPI2
USART1
USART2
USART3
USART1 Mode and Configuration
Mode
Mode Asynchronous
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
DMA Request
Channel
Direction
Priority
USART1_RX
DMA1 Channel 5
Peripheral To Memory
Low
USART1_TX
DMA1 Channel 4
Memory To Peripheral
Low
Add
Delete
DMA Request Settings
Peripheral
Memory
Mode Normal
Increment Address
Data Width
Byte
Byte
Pinout view
System view
RCC_OSC_IN
RCC_OSC_OUT


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Project Manager
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Project
Code Generator
Advanced Settings
Project Settings
Project Name
text2_2
Project Location
D:\work\text2_2
Browse
Application Structure
Advanced
Do not generate the main()
Toolchain Folder Location
D:\work\text2_2\text2_2\
Toolchain / IDE
MDK-ARM
Min Version
V5.32
Generate Under Root
Linker Settings
Minimum Heap Size
0x200
Minimum Stack Size
0x400
Thread-safe Settings
Cortex-M3NS
Enable multi-threaded support
Thread-safe Locking Strategy
Default – Mapping suitable strategy depending on RTOS selection.
Mcu and Firmware Package
Mcu Reference
STM32F103C8Tx
Firmware Package Name and Version
STM32Cube FW_F1 V1.8.7
Use latest available version
Use Default Firmware Location


STM32CubeMX Untitled*: STM32F103C8Tx
File
Window
Help
Home
STM32F103C8Tx
Untitled - Clock Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Resolve Clock Issues
LSI RC
40
40 KHz
To IWDG (KHz)
To FLITFCLK (MHz)
HSI RC
8
8 MHz
System Clock Mux
HSI
HSE
PLLCLK
SYSCLK (MHz)
72
AHB Prescaler
/ 1
HCLK (MHz)
72 MHz max
APB1 Prescaler
/ 2
APB2 Prescaler
/ 1
PCLK1
36 MHz max
PCLK2
72 MHz max
HCLK to AHB bus, core, memory and DMA (MHz)
To Cortex System timer (MHz)
FCLK (MHz)
APB1 peripheral clocks (MHz)
APB1 Timer clocks (MHz)
APB2 peripheral clocks (MHz)
APB2 timer clocks (MHz)
X 2
X 1
Enable CSS
PLL Source Mux
/ 2
PLL
X 9
USB Prescaler
To USB (MHz)
ADC Prescaler
To ADC1,2
Input frequency
4-16 MHz



(2) 代码编写

在`main.c`中编写以下代码：

```
/* USER CODE BEGIN PV */
uint8_t tx_buffer[] = "hello windows!\r\n";
/* USER CODE END PV */

/* USER CODE BEGIN WHILE */
while (1)
{
    // 使用DMA方式发送，不占用CPU
    HAL_UART_Transmit_DMA(&huart1, tx_buffer, strlen((char*)tx_buffer));
    HAL_Delay(500); // 延时控制发送频率
    /* USER CODE END WHILE */
}
/* USER CODE END 3 */

/* USER CODE BEGIN 4 */
// DMA发送完成回调函数（可选）
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        // 发送完成后的处理逻辑
    }
}
/* USER CODE END 4 */
```

完整代码

```
/* USER CODE BEGIN Header */
```

```c
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <string.h>  // 需要用到 strlen 函数
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
UART_HandleTypeDef huart1;
DMA_HandleTypeDef hdma_usart1_rx;
DMA_HandleTypeDef hdma_usart1_tx;

/* USER CODE BEGIN PV */
uint8_t tx_buffer[] = "hello windows!\r\n";  // DMA发送的数据缓存
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
```

```c
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  // 这里不需要额外初始化，DMA在MX_DMA_Init中已配置好
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    // 使用DMA方式发送数据（CPU不参与搬运，效率极高）
    HAL_UART_Transmit_DMA(&huart1, tx_buffer, strlen((char*)tx_buffer));

    // 【必须加延时！】
```

```
    // 如果不加延时，DMA会在几毫秒内塞入成百上千帧数据，导致Win10串口助手卡死（假死）。
    HAL_Delay(500);

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief USART1 Initialization Function
  * @param None
  * @retval None
```

```c
  */
static void MX_USART1_UART_Init(void)
{

  /* USER CODE BEGIN USART1_Init 0 */

  /* USER CODE END USART1_Init 0 */

  /* USER CODE BEGIN USART1_Init 1 */

  /* USER CODE END USART1_Init 1 */
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART1_Init 2 */

  /* USER CODE END USART1_Init 2 */

}

/**
  * Enable DMA controller clock
  */
static void MX_DMA_Init(void)
{

  /* DMA controller clock enable */
  __HAL_RCC_DMA1_CLK_ENABLE();

  /* DMA interrupt init */
  /* DMA1_Channel4_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA1_Channel4_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel4_IRQn);
  /* DMA1_Channel5_IRQn interrupt configuration */
  HAL_NVIC_SetPriority(DMA1_Channel5_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel5_IRQn);

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
```

```c
static void MX_GPIO_Init(void)
{
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
/*
 * 可选：DMA发送完成回调函数。如果你不需要在发送完成后执行特殊操作，可以不写。
 * 这个函数会在DMA把所有数据搬运到串口后自动被调用。
 */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        // 发送完成后的处理逻辑（比如翻转LED指示灯）
    }
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
```

```
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

(3)下载编译

步骤跟上述一样

(4) 调用串口助手，查看结果

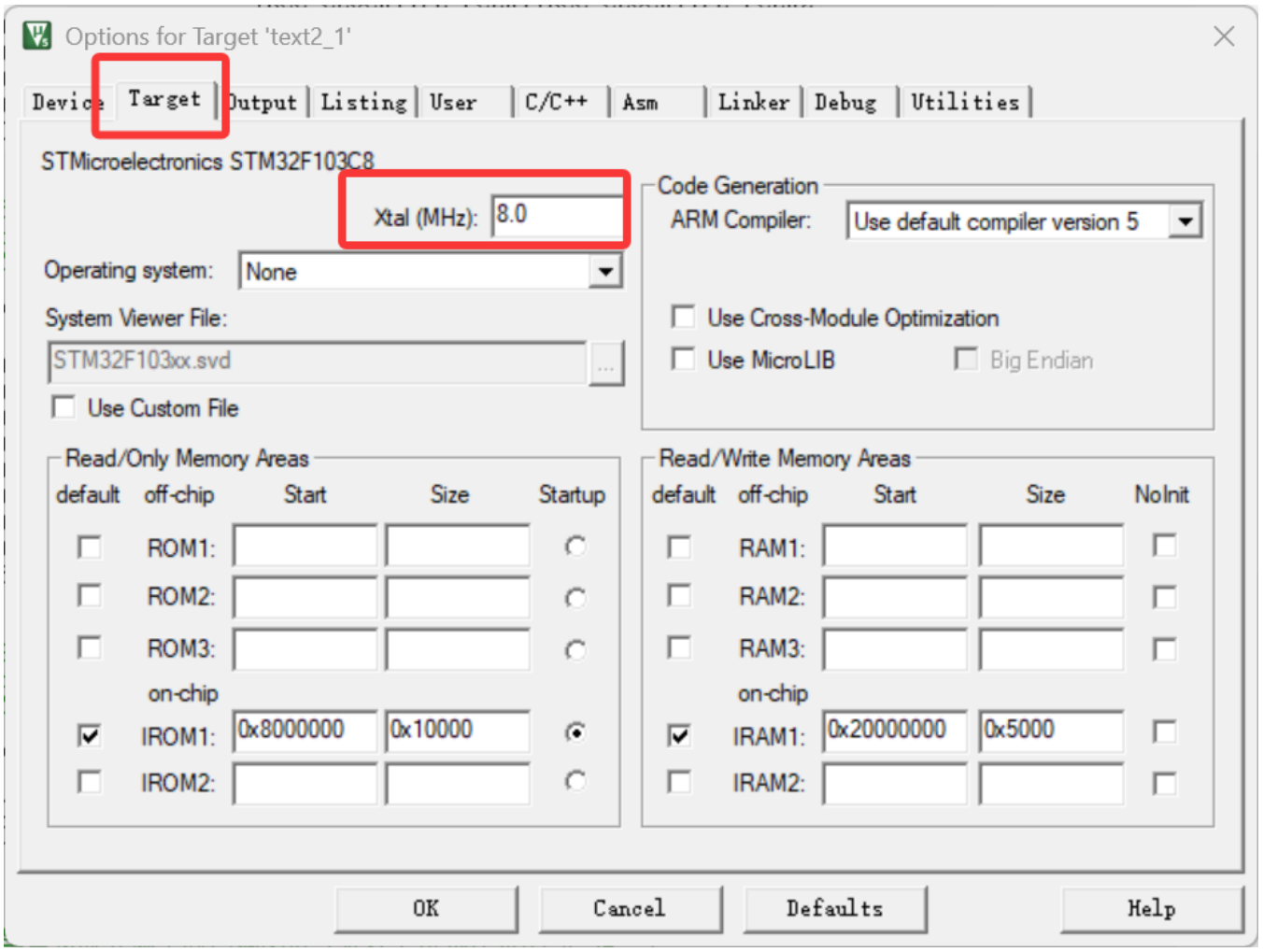


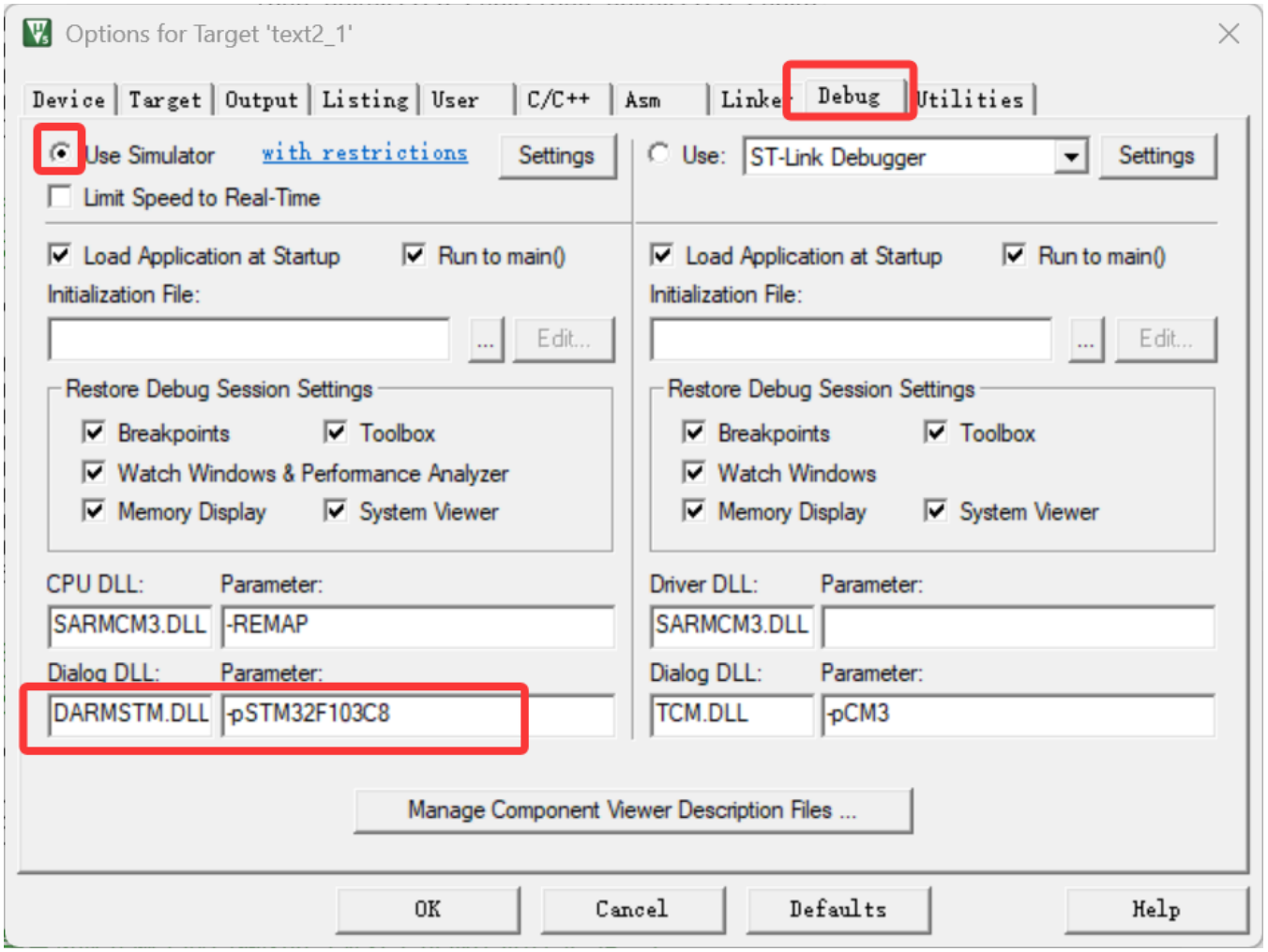
五、任务三

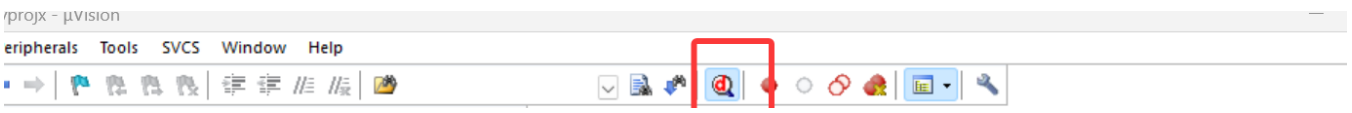
1.实验目的

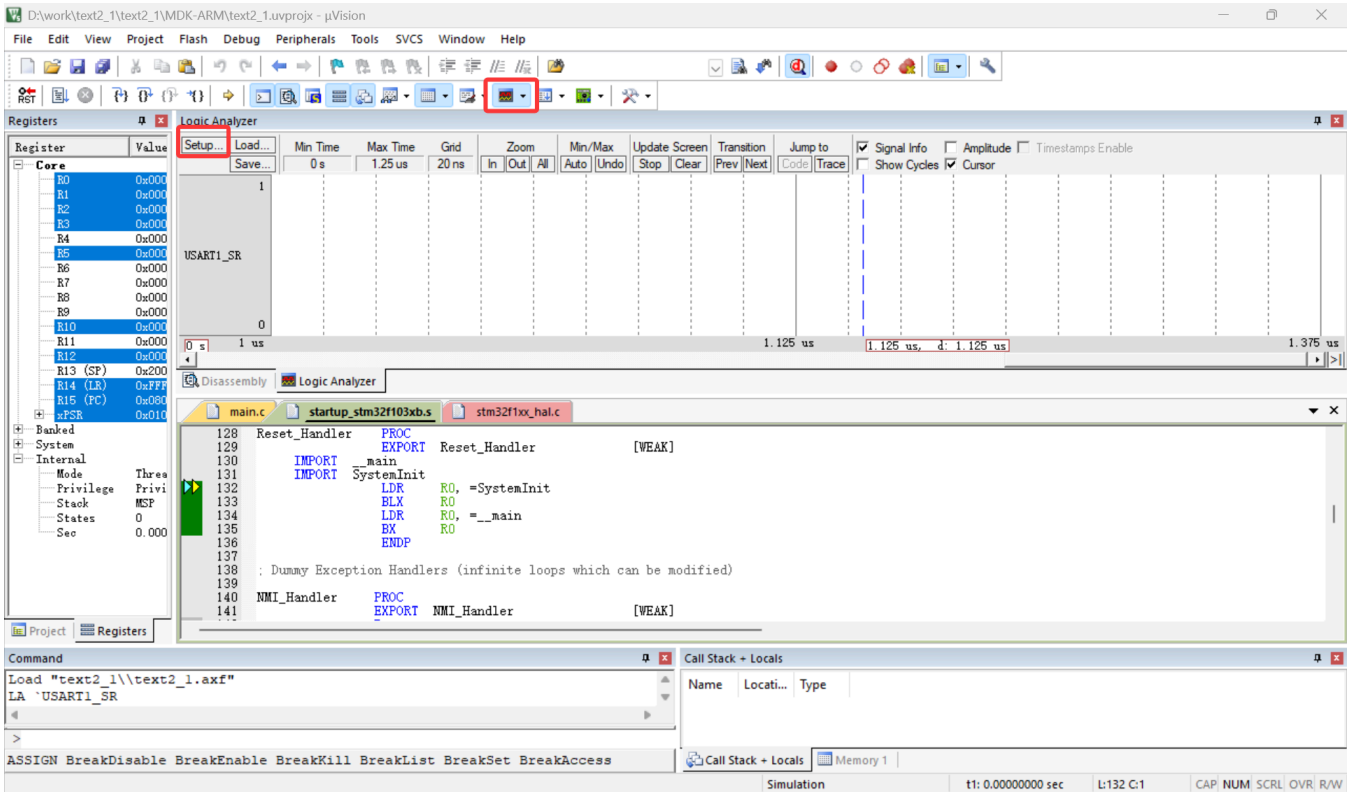
在没有示波器条件下，可以使用Keil的软件仿真逻辑分析仪功能观察任务二（1）串口输出波形，并分析时序状态正确与否，计算波特率实际为多少。

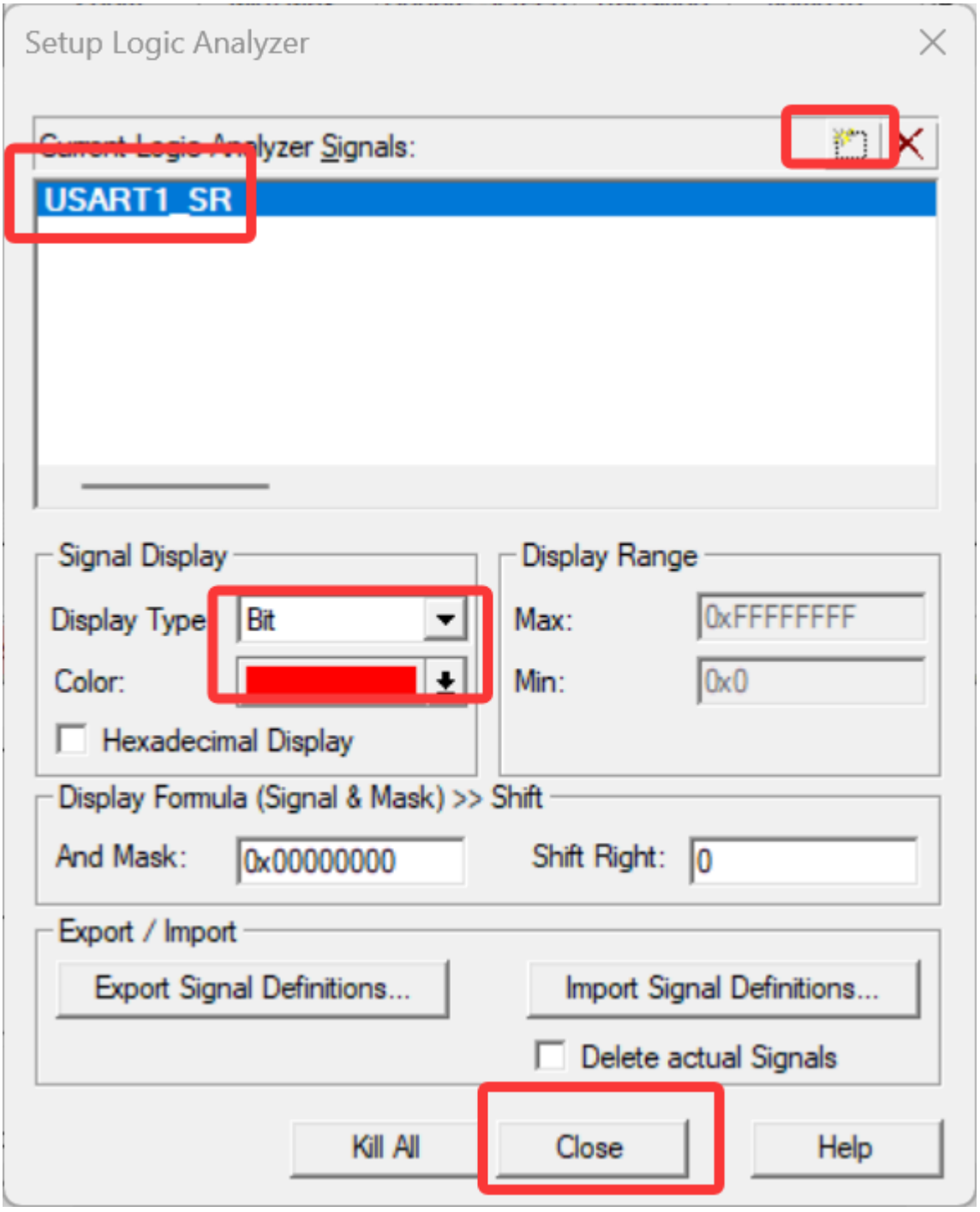
2.环境搭建

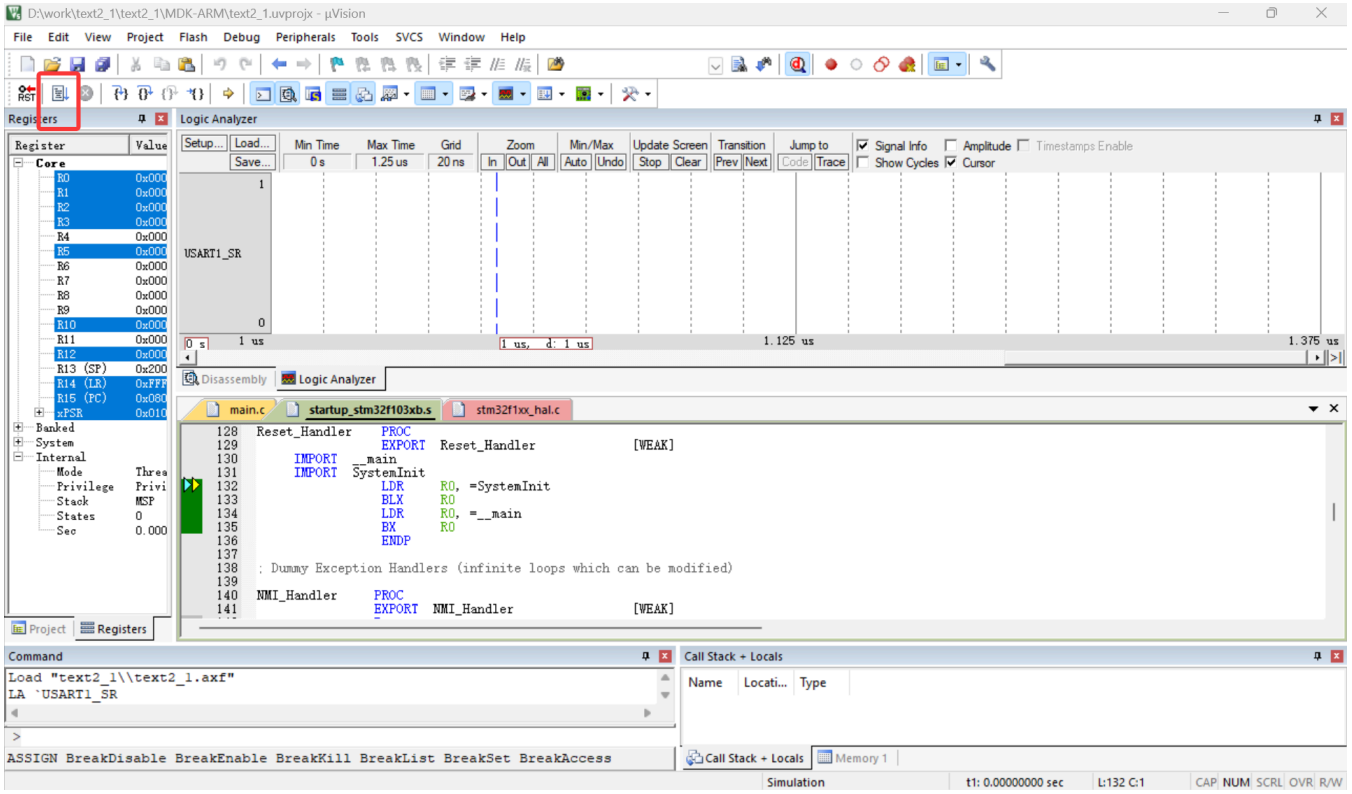

Options for Target 'text2_1'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
STMicroelectronics STM32F103C8
Xtal (MHz): 8.0
Operating system: None
System Viewer File:
STM32F103xx.svd
Use Custom File
Code Generation
ARM Compiler: Use default compiler version 5
Use Cross-Module Optimization
Use MicroLIB
Big Endian
Read/Only Memory Areas
default off-chip Start Size Startup
ROM1:
ROM2:
ROM3:
on-chip
IROM1: 0x8000000 0x10000
IROM2:
Read/Write Memory Areas
default off-chip Start Size NoInit
RAM1:
RAM2:
RAM3:
on-chip
IRAM1: 0x20000000 0x5000
IRAM2:
OK
Cancel
Defaults
Help


Options for Target 'text2_1'
Device
Target
Output
Listing
User
C/C++
Asm
Linker
Debug
Utilities
Use Simulator
with restrictions
Settings
Use:
ST-Link Debugger
Settings
Limit Speed to Real-Time
Load Application at Startup
Run to main()
Initialization File:
Edit...
Restore Debug Session Settings
Breakpoints
Toolbox
Watch Windows & Performance Analyzer
Watch Windows
Memory Display
System Viewer
CPU DLL:
Parameter:
SARMCM3.DLL
-REMAP
Dialog DLL:
DARMSTM.DLL
-pSTM32F103C8
Driver DLL:
TCM.DLL
-pCM3
Manage Component Viewer Description Files ...
OK
Cancel
Defaults
Help

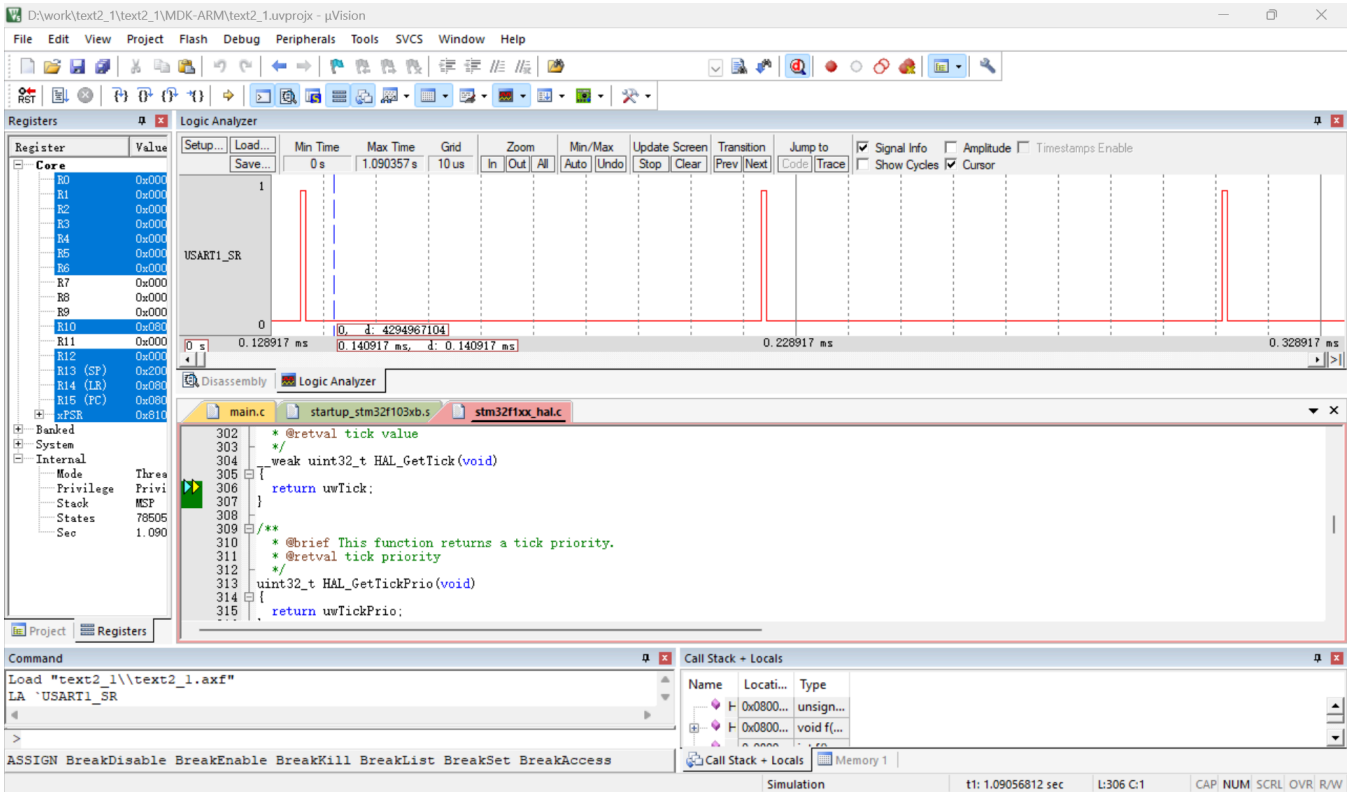

eripherals
Tools
SVCS
Window
Help


File
Edit
View
Project
Flash
Debug
Peripherals
Tools
SVCS
Window
Help
Registers
Logic Analyzer
Setup...
Load...
Save...
USART1_SR
Disassembly
Logic Analyzer
main.c
startup_stm32f103xb.s
stm32f1xx_hal.c
Reset_Handler PROC
EXPORT Reset_Handler [WEAK]
IMPORT __main
IMPORT SystemInit
LDR R0, =SystemInit
BLX R0
LDR R0, =__main
BX R0
ENDP
; Dummy Exception Handlers (infinite loops which can be modified)
NMI_Handler PROC
EXPORT NMI_Handler [WEAK]
Command
Load "text2_1\\text2_1.axf"
LA `USART1_SR
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess
Call Stack + Locals
Name
Locati...
Type
Memory 1
Simulation
t1: 0.00000000 sec
L:132 C:1
CAP
NUM
SCRL
OVR
R/W

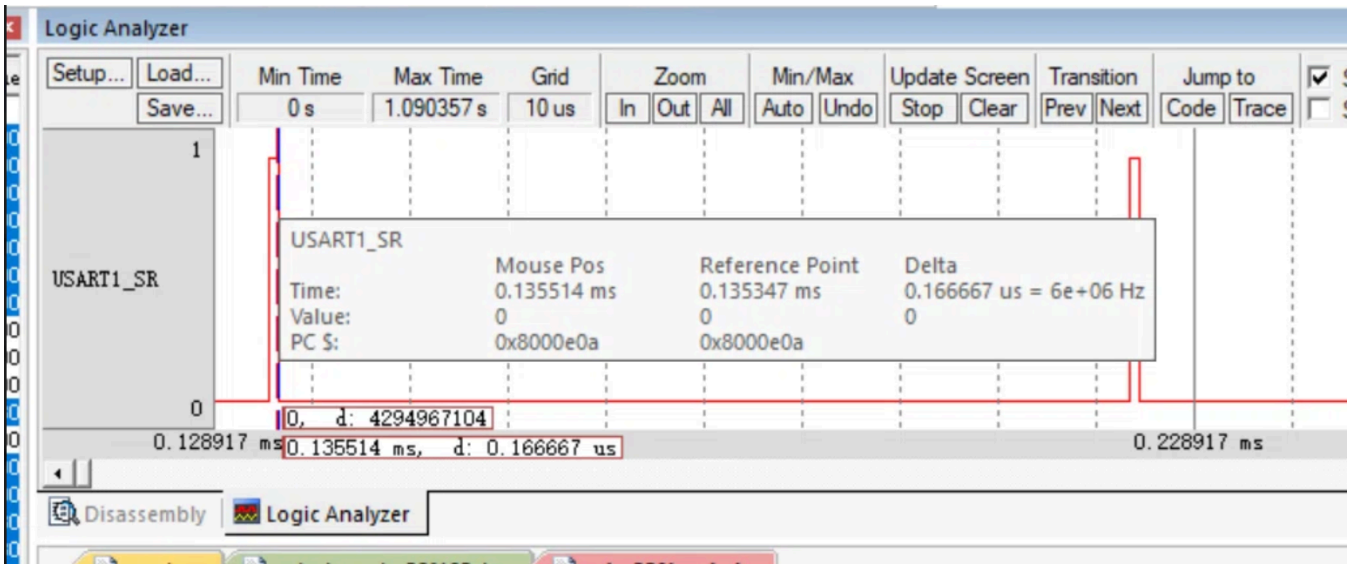

Setup Logic Analyzer
Current Logic Analyzer Signals:
USART1_SR
Signal Display
Display Type
Bit
Color:
Hexadecimal Display
Display Range
Max:
0xFFFFFFFF
Min:
0x0
Display Formula (Signal & Mask) >> Shift
And Mask:
0x00000000
Shift Right:
0
Export / Import
Export Signal Definitions...
Import Signal Definitions...
Delete actual Signals
Kill All
Close
Help

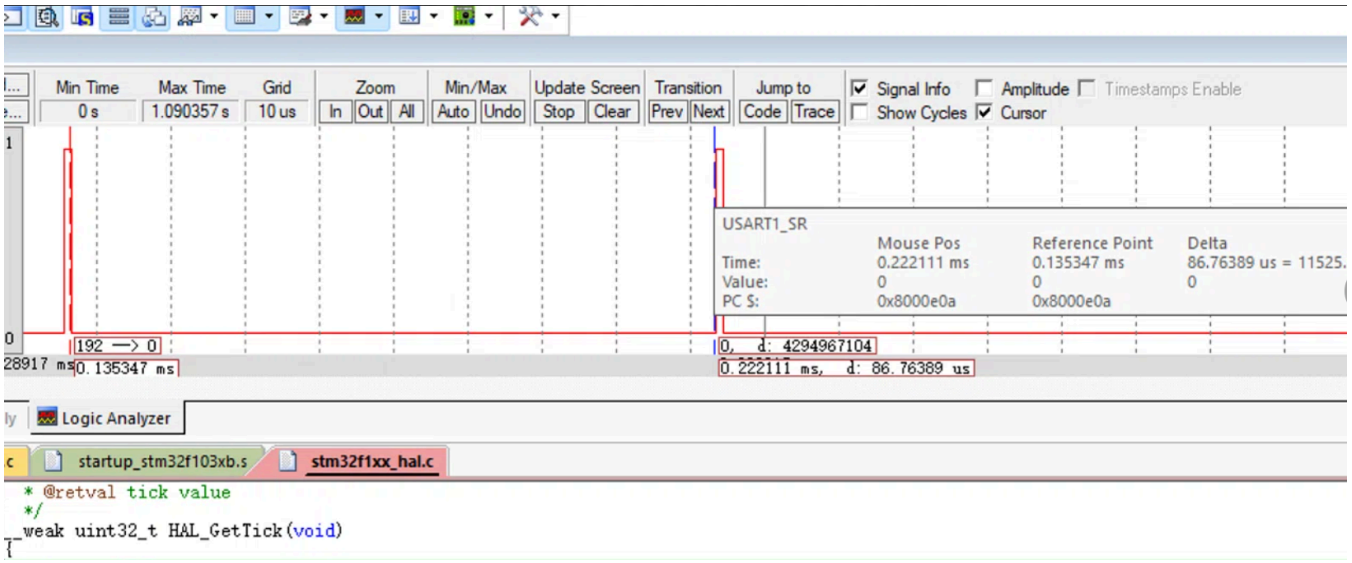

D:\work\text2_1\text2_1\MDK-ARM\text2_1.uvprojx - µVision
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Logic Analyzer
USART1_SR
Disassembly
Logic Analyzer
main.c
startup_stm32f103xb.s
stm32f1xx_hal.c
Reset_Handler PROC
EXPORT Reset_Handler [WEAK]
IMPORT __main
IMPORT SystemInit
LDR R0, =SystemInit
BLX R0
LDR R0, =__main
BX R0
ENDP
; Dummy Exception Handlers (infinite loops which can be modified)
NMI_Handler PROC
EXPORT NMI_Handler [WEAK]
Command
Load "text2_1\\text2_1.axf"
LA `USART1_SR
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess
Call Stack + Locals
Name Locati... Type
Simulation
t1: 0.00000000 sec
L:132 C:1

3.仿真波形


D:\work\text2_1\text2_1\MDK-ARM\text2_1.uvprojx - µVision
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Logic Analyzer
USART1_SR
Disassembly
Logic Analyzer
main.c
startup_stm32f103xb.s
stm32f1xx_hal.c
* @retval tick value
*/
__weak uint32_t HAL_GetTick(void)
{
return uwTick;
}
/**
* @brief This function returns a tick priority.
* @retval tick priority
*/
uint32_t HAL_GetTickPrio(void)
{
return uwTickPrio;
Command
Load "text2_1\\text2_1.axf"
LA `USART1_SR
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess
Call Stack + Locals
Name Locati... Type
Simulation
t1: 1.09056812 sec
L:306 C:1

4.结果分析





A.波形观测现象

- **观测信号**：当前抓取的是 `USART1_SR`（状态寄存器），它反映的是串口发送过程中标志位（如TXE/TC）的跳变。
- **时间间隔**：两个脉冲之间的间隔约为500ms，这与代码中 `HAL_Delay(500)` 的控制逻辑完全一致。
- **逻辑分析仪悬浮窗数据**：
  - `Delta = 86.76389 us`（即一帧数据的传输时间）。
  - 软件显示的 `11525.5 Hz` 是“帧频率”，并非波特率。

B.时序状态分析（正确与否）

根据 UART 异步串行通信协议（8N1模式：8位数据位，无校验，1位停止位），一帧标准数据包含**10个位**：

1. **起始位 (1 bit)**：逻辑低电平（0）。总线空闲时保持高电平，检测到下降沿代表一帧开始。
2. **数据位 (8 bits)**：低位在前，高位在后。传送具体字符。
3. **停止位 (1 bit)**：逻辑高电平（1）。代表该字节传送结束。
4. **空闲状态**：停止位结束后，总线回到高电平。

- **结论**：你的代码配置为 `BaudRate = 115200, WordLength = 8B, StopBits = 1, Parity = None`。根据测得的时序，波形清晰反映了每帧数据占据10个位周期的时间，**时序状态完全正确。**

C. 波特率实际值计算（实验报告核心部分）

已知数据：

- 一帧数据总时间（10个位）Tframe=86.76389 μs$T\boldsymbol{fra}me$=86.76389 μ**s
- 理论波特率 Baudtheory=115200 bps$B\boldsymbol{a}u\boldsymbol{d}t\boldsymbol{h}eory$=115200 $b\boldsymbol{p}s$
- 系统时钟配置：PCLK2 = 72 MHz

计算步骤：

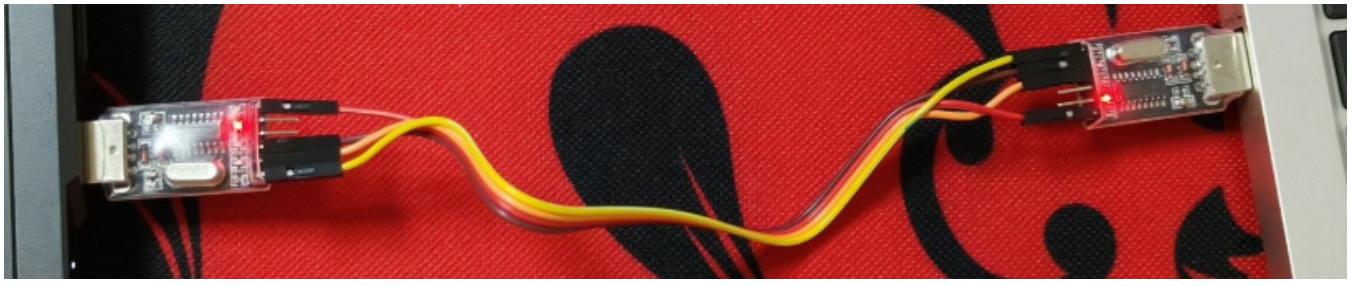
1. **计算单个位（1 bit）的实际传输时间**：
   Tbit=Tframe10=86.76389 μs10=8.676389 μs$T\boldsymbol{bi}t$=10$T\boldsymbol{fra}me$=1086.76389 μ**s=8.676389 μ**s
2. **计算实际波特率**：
   Baudactual=1Tbit=18.676389×10−6 s≈115254.4 bps$B\boldsymbol{a}u\boldsymbol{d}a\boldsymbol{ctu}a$**$l$=$T\boldsymbol{bi}t$1=8.676389×10−6 s1≈115254.4 $b\boldsymbol{p}s$
3. **计算误差率**：
   Error=|115254.4−115200|115200×100%≈0.047%E**$rror$=115200|115254.4−115200|×100%≈0.047%

D.**分析结论**：
实际波特率为 **115254.4 bps**，与理论值 115200 bps 的误差仅为 **0.047%。**在串口通信中，通常允许的波特率误差范围为 **±3%** 以内，甚至苛刻环境下允许±2%。你的误差远远低于这个阈值。
这得益于你之前配置的 **72MHz 系统时钟**（8MHz外部晶振 × PLL 9倍频）。该时钟频率下，USART 的分频系数计算精度极高，因此**通信质量非常稳定，上位机接收不会出现乱码或丢包。**

六、 串口传输文件的实验

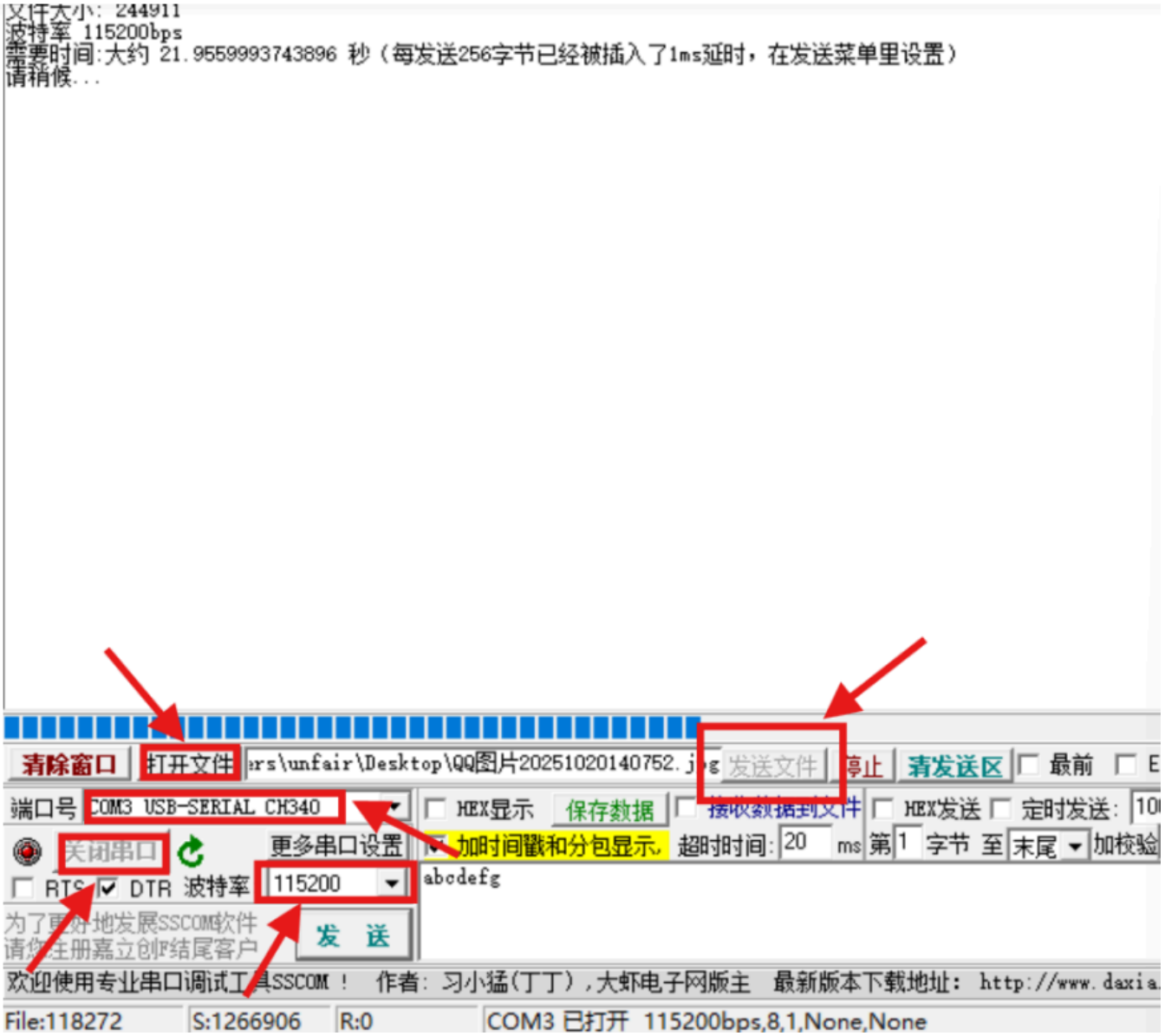
1.硬件连接

将两个“USB/TTL”用杜邦线正确连接，这里我将TXD连接RXD，RXD连接TXD，5V连接5V，GND连接GND ，两USB接口插电脑



2.sscom调试

连接到两台计算机后，在两台电脑上都打开串口助手，这我这里进行发送文件，我这里选择了一张图片
发送端：根据步骤选择



接收端：

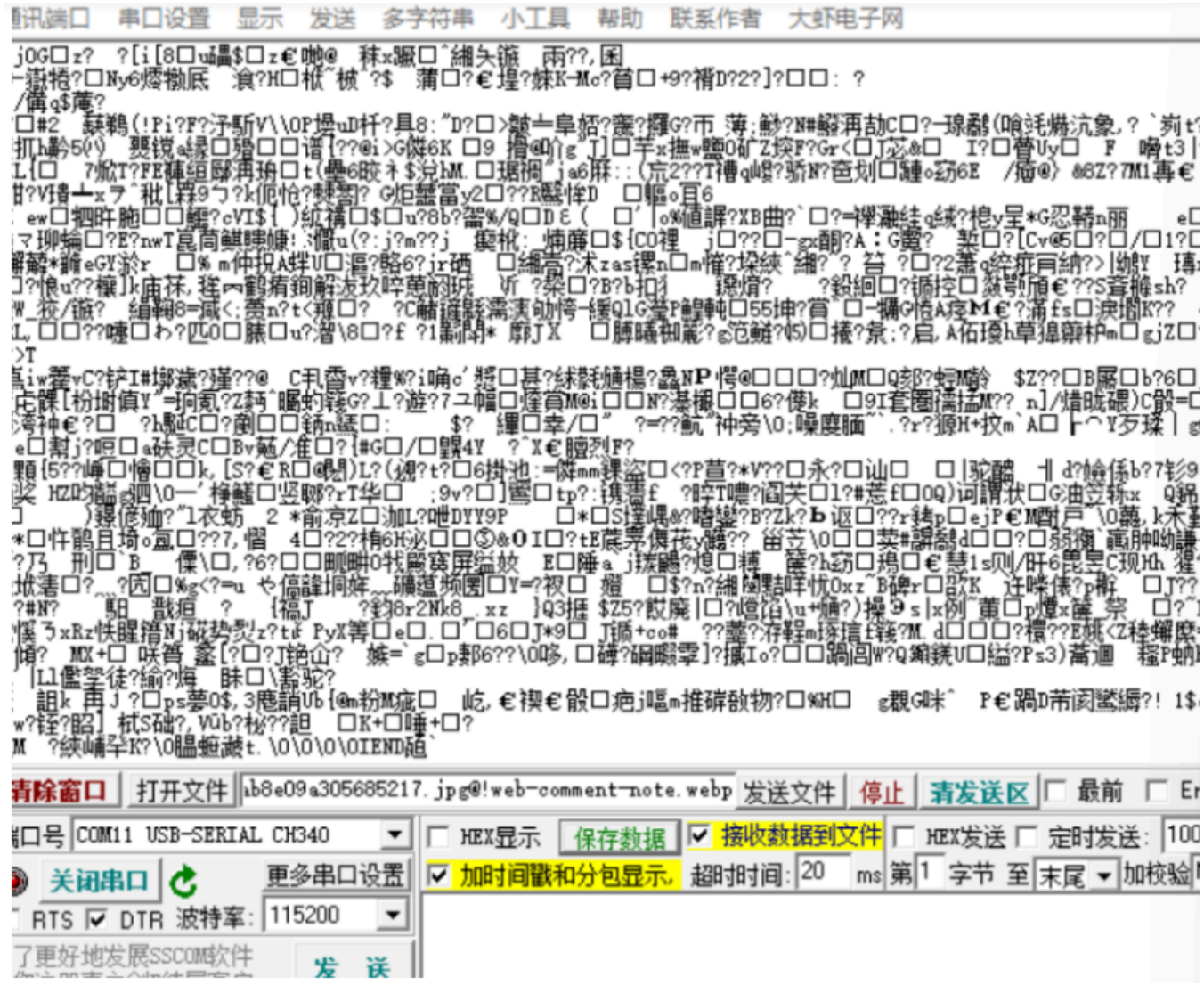


3.文件发送与接收

传输时间约为21.956秒


SSCOM V5.13.1 串口/网络数据调试器,作者:大虾丁丁,2618058@qq.com. QQ群: 52502449
通讯端口 串口设置 显示 发送 多字符串 小工具 帮助
文件大小: 244911
波特率 115200bps
需要时间:大约 21.9559993743896 秒（每发送256字节已经被插入了1ms延时，在发送菜单里设置）
请稍候...
清除窗口
打开文件
发送文件
停止
清发送区
最前
English
保存参数
扩展
端口号
COM3 USB-SERIAL CH340
HEX显示
保存数据
HEX发送
定时发送:
1000
ms/次
加回车换行
关闭串口
更多串口设置
加时间戳和分包显示,
超时时间:
20
ms
第
1
字节 至
末尾
加校验
None

接收



点击保存数据可以看到存储的位置

在这里发送的是一个图片jpg格式，接收方收到的是dat格式，将后缀改为jpg即可打开



4.结果分析

（1）文件大小、波特率和传输时间的关系

文件大小、波特率和传输时间的关系

串口采用异步通信，8N1 格式下每传输 1 个字节（8 位数据）实际需发送 10 位（1 起始位 + 8 数据位 + 1 停止位）。理论传输时间公式为：

理论时间=文件字节数×10/波特率

本实验文件 244,911 字节，波特率 115,200 bps：

理论时间=244,911×10/115,200≈21.26

SSCOM 估算时间为 21.96 秒，实际传输时间略长于理论值，主要原因包括：SSCOM 每发送 256 字节插入 1ms 延时（累计约 0.96 秒），以及 PC 端文件读写、界面刷新和系统调度带来的额外开销。传输时间与文件大小成正比，与波特率成反比。

（2）只接 TX/RX，不接 GND 能否正常工作

不能正常工作。串口 TX/RX 是单端信号，逻辑电平以 GND 为参考。两个模块若不共地，接收端无法正确判断信号线上的电压高低，会导致数据无法接收或收到乱码。电源线（VCC）可以不接，因为两个模块各自通过 USB 供电；但 GND 必须连接，以提供共同的电压参考基准。

七、总结

本次实验基于STM32F103与HAL库，分别利用串口中断与DMA方式实现了与上位机的通信。在中断任务中，通过收发回调函数及状态标志位成功实现了“#”暂停与“*”恢复发送的功能；在DMA任务中，验证了其不占用CPU的高速传输特性。结合Keil逻辑分析仪进行时序观测，实测波特率为115254.4bps，误差仅0.047%，完美验证了72MHz系统时钟的准确性。通过实验，不仅掌握了外设底层配置与代码调试技巧，更深刻理解了HAL库的设计思想及阻塞、中断、DMA三种通信方式的本质区别。